



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA HÀ NỘI

BÁO CÁO CUỐI KỲ

MỘT SỐ VẤN ĐỀ CHỌN LỌC VỀ THỊ GIÁC MÁY TÍNH
(ADVANCED READING ON COMPUTER VISION)

HỆ THỐNG GIÁM SÁT GIAO THÔNG THÔNG MINH VÀ NHẬN DIỆN BIỂN SỐ XE VIỆT NAM

KẾT HỢP PHÁT HIỆN ĐỐI TƯỢNG YOLOv8, THEO DÕI ĐA ĐỐI TƯỢNG
BYTETRACK,
PHÁT HIỆN VI PHẠM GIAO THÔNG VÀ NHẬN DẠNG BIỂN SỐ VỚI PADDLEOCR

Nhóm thực hiện:

Nguyễn Hải Đăng -

[22001560]

Chữ Văn Hiệp - [22001576]

Nguyễn Mậu Đức -

[22001568]

Giảng viên hướng dẫn:

TS. Cao Văn Chung

Hà Nội, tháng 1 năm 2026

Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành nhất đến **Tiến sĩ Cao Văn Chung** – giảng viên hướng dẫn môn học “Một số vấn đề chọn lọc về Thị giác máy tính”. Thầy đã tận tình hướng dẫn, định hướng nghiên cứu và cung cấp những kiến thức chuyên môn quý báu trong suốt quá trình thực hiện đề tài.

Chúng em cũng xin cảm ơn **Khoa Toán – Cơ – Tin học, Trường Đại học Khoa học Tự nhiên**, Đại học Quốc gia Hà Nội đã tạo điều kiện về cơ sở vật chất, tài liệu học tập và môi trường nghiên cứu thuận lợi để nhóm hoàn thành đề tài này.

Đồng thời, nhóm xin bày tỏ lòng biết ơn đến các bạn sinh viên cùng lớp đã đóng góp ý kiến, chia sẻ kinh nghiệm và hỗ trợ trong quá trình thu thập dữ liệu, thử nghiệm hệ thống.

Cuối cùng, nhóm xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, khích lệ trong suốt quá trình học tập và nghiên cứu.

Mặc dù đã cố gắng hết sức, báo cáo không tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ quý thầy cô và các bạn để hoàn thiện hơn nữa kiến thức và kỹ năng nghiên cứu.

Hà Nội, tháng 1 năm 2026
Nhóm thực hiện

Tóm tắt

Tổng quan

Đề tài “**Hệ thống giám sát giao thông thông minh và nhận diện biển số xe Việt Nam**” trình bày việc xây dựng một hệ thống hoàn chỉnh ứng dụng các kỹ thuật Thị giác máy tính và Học sâu vào bài toán giám sát giao thông thực tế tại Việt Nam.

Mục tiêu nghiên cứu

Hệ thống được phát triển với các mục tiêu chính:

1. **Phát hiện và phân loại phương tiện:** Nhận diện 5 loại phương tiện phổ biến trong giao thông Việt Nam (xe máy, ô tô, xe buýt, xe tải, xe đạp) sử dụng mô hình YOLOv8.
2. **Theo dõi đa đối tượng:** Ứng dụng thuật toán ByteTrack để theo dõi quỹ đạo di chuyển và đếm lưu lượng xe theo từng làn đường.
3. **Phát hiện vi phạm giao thông:** Tự động phát hiện các hành vi vượt đèn đỏ và đi sai làn đường thông qua phân tích hướng di chuyển và logic hình học.
4. **Nhận diện biển số xe:** Xây dựng pipeline hoàn chỉnh phát hiện và đọc biển số xe Việt Nam sử dụng YOLOv8 kết hợp PaddleOCR.

Phương pháp

- **Phát hiện đối tượng:** YOLOv8n (Anchor-free, C2f Module) được huấn luyện trên dữ liệu giao thông Việt Nam tự thu thập với hơn 16.000 ảnh và 65.000 bounding box.
- **Theo dõi:** ByteTrack với cơ chế liên kết hai giai đoạn (high-score và low-score matching) kết hợp Bộ lọc Kalman.
- **Xác định hướng:** Direction Fusion kết hợp Trajectory Analysis (Reference Vector) và ROI-based Direction.
- **Nhận dạng biển số:** CLAHE + Sharpening tiền xử lý, PaddleOCR (CRNN + CTC) cho OCR, cùng cơ chế Voting & Lock để ổn định kết quả.

Kết quả chính

- **Phát hiện phương tiện:** mAP@50 đạt **86.1%**, trong đó lớp “Ô tô” đạt 92.2%.

- **Phát hiện biển số:** mAP@50 đạt **99.5%**, Precision = 99.97%.
- **Tốc độ xử lý:** 25–30 FPS với pipeline giám sát, 5–10 FPS với full pipeline bao gồm OCR.
- **Phát hiện vi phạm:** Hệ thống hoạt động ổn định với logic phát hiện vượt đèn đỏ và sai làn đường.

Ý nghĩa

Đề tài chứng minh khả năng ứng dụng các kỹ thuật AI tiên tiến vào giám sát giao thông Việt Nam với chi phí thấp, tốc độ thời gian thực, mở ra tiềm năng triển khai rộng rãi trong quản lý đô thị thông minh.

Từ khóa: YOLOv8, ByteTrack, Phát hiện vi phạm giao thông, Nhận diện biển số, PaddleOCR, Thị giác máy tính

Mục lục

Lời cảm ơn	ii
Tóm tắt	iii
Danh sách các từ viết tắt	xi
1 Mở đầu	1
1.1 Đặt vấn đề	1
1.1.1 Bối cảnh giao thông Việt Nam năm 2025	1
1.1.2 Bài toán nhận diện biển số xe	1
1.1.3 Định hướng nghiên cứu	2
1.2 Mục tiêu của đề tài	2
1.2.1 Module phát hiện và theo dõi phương tiện	2
1.2.2 Module phát hiện vi phạm	2
1.2.3 Module nhận diện biển số xe	2
1.3 Phạm vi nghiên cứu	3
1.3.1 Dữ liệu	3
1.3.2 Công nghệ sử dụng	3
1.3.3 Giới hạn	3
1.4 Cấu trúc báo cáo	3
2 Cơ sở lý thuyết: Phát hiện và theo dõi đối tượng	5
2.1 Tổng quan về bài toán Phát hiện đối tượng	5
2.1.1 Giới thiệu chung	5
2.1.2 Định nghĩa toán học	5
2.1.3 Thách thức của bài toán	6
2.2 Cơ sở lý thuyết về mô hình YOLO	6
2.2.1 Hợp nhất bài toán thành một mạng duy nhất	6
2.2.2 Cơ chế chia lưới và Biểu diễn đầu ra	6
2.2.3 Kiến trúc mạng tổng quát	7
2.2.4 Sự tiến hóa và Lựa chọn YOLOv8	8
2.3 Mô hình YOLOv8 chi tiết	8
2.3.1 Cơ chế Anchor-free	8
2.3.2 Các thành phần kiến trúc đặc trưng	9
2.3.3 Hàm mất mát (Loss Function)	10
2.4 Các độ đo đánh giá (Evaluation Metrics)	10
2.4.1 Precision và Recall	10

2.4.2	Average Precision (AP)	11
2.4.3	Mean Average Precision (mAP)	11
2.5	Theo dõi đa đối tượng (Multi-Object Tracking)	11
2.5.1	Bộ lọc Kalman (Kalman Filter)	11
2.5.2	Liên kết dữ liệu (Data Association)	13
2.5.3	Thuật toán ByteTrack	13
2.6	Xử lý ảnh và Hình học tính toán	16
2.6.1	Không gian màu HSV trong nhận diện tín hiệu đèn	16
2.6.2	Hình học tính toán trong phát hiện vi phạm	17
3	Cơ sở lý thuyết: Nhận diện biển số xe	19
3.1	Tổng quan về ALPR	19
3.2	Biển số xe Việt Nam	19
3.2.1	Các định dạng biển số	19
3.2.2	Thách thức nhận diện biển số Việt Nam	20
3.3	PaddleOCR - Nhận dạng ký tự	20
3.3.1	Giới thiệu PaddleOCR	20
3.3.2	Kiến trúc CRNN	20
3.3.3	CTC Loss	20
3.4	Tiền xử lý ảnh biển số	21
3.4.1	CLAHE - Cân bằng histogram thích ứng	21
3.4.2	Sharpening - Làm sắc nét	21
3.4.3	Quy trình tiền xử lý hoàn chỉnh	22
3.5	So sánh các phiên bản YOLOv8	22
3.6	Cơ chế Validation & Retry	23
3.6.1	Vấn đề với OCR đơn frame	23
3.6.2	Giải pháp: Format Validation	23
3.6.3	Retry Mechanism	23
3.7	Character Correction cho biển số Việt Nam	24
3.7.1	Các lỗi phổ biến của OCR	24
3.7.2	Logic hiệu chỉnh dựa trên định dạng	24
3.8	Hai phương pháp triển khai ALPR	24
3.8.1	Phương pháp 1: Two-Stage ALPR (Hai giai đoạn)	24
3.8.2	Phương pháp 2: One-Stage Semi-supervised (Một giai đoạn)	25
3.8.3	So sánh hai phương pháp	26
4	Phương pháp thực hiện	27
4.1	Kiến trúc tổng thể của hệ thống	27
4.2	Chuẩn bị dữ liệu phát hiện phương tiện	29
4.2.1	Quy trình thu thập dữ liệu	29
4.2.2	Tiền xử lý dữ liệu	30
4.3	Huấn luyện mô hình phát hiện phương tiện	30
4.3.1	Cấu hình huấn luyện	30
4.3.2	Quá trình tối ưu hóa	31
4.4	Thuật toán xác định hướng di chuyển	31
4.4.1	Phân tích quỹ đạo với Reference Vector (Trajectory-based)	31
4.4.2	Xác định hướng dựa trên vùng quan tâm (ROI-based)	31

4.4.3	Cơ chế kết hợp (Direction Fusion)	31
4.5	Hệ thống phát hiện vi phạm	32
4.5.1	Quản lý tín hiệu đèn giao thông	32
4.5.2	Logic phát hiện vượt đèn đỏ	32
4.5.3	Logic phát hiện sai làn đường	32
4.5.4	Điểm chạm tham chiếu (Reference Point)	33
4.5.5	Ngưỡng dung sai	33
4.6	Pipeline nhận diện biển số xe (ALPR)	34
4.6.1	Phương pháp 1: Mô hình phát hiện biển số chuyên biệt	34
4.6.2	Phương pháp 2: Tích hợp phát hiện đa lớp (Semi-supervised)	35
4.7	Tiền xử lý ảnh biển số	37
4.7.1	Bước 1: CLAHE (Contrast Limited Adaptive Histogram Equalization)	37
4.7.2	Bước 2: Sharpening	37
4.8	OCR với PaddleOCR	37
4.8.1	Cấu hình PaddleOCR	37
4.8.2	Character Correction	37
4.9	Validation & Retry Mechanism	38
4.9.1	Vấn đề	38
4.9.2	Giải pháp: Format Validation	38
4.9.3	Retry Mechanism	38
4.10	Tích hợp với Vehicle Tracking	39
4.11	Tổng kết phương pháp	39
5	Thực nghiệm và Đánh giá kết quả	40
5.1	Môi trường thực nghiệm	40
5.1.1	Cấu hình phần cứng	40
5.1.2	Cấu hình phần mềm	40
5.2	Kết quả huấn luyện mô hình phát hiện phương tiện	41
5.2.1	Thiết lập huấn luyện	41
5.2.2	Phân tích quá trình hội tụ	41
5.2.3	Kết quả định lượng	43
5.2.4	Phân tích lỗi qua ma trận nhầm lẫn	43
5.2.5	Đánh giá định tính	43
5.3	Kết quả huấn luyện Plate Detection	44
5.3.1	Quá trình huấn luyện	44
5.3.2	Kết quả cuối cùng	44
5.3.3	Ma trận nhầm lẫn Plate Detection	44
5.4	Kết quả huấn luyện Mô hình tích hợp (6 lớp)	47
5.4.1	Kết quả định lượng	47
5.4.2	Đánh giá trực quan	48
5.5	Kết quả Inference thực tế	48
5.5.1	Môi trường thử nghiệm	48
5.5.2	Đánh giá Phương pháp 1 (Two-Stage: Model chuyên biệt)	49
5.5.3	Đánh giá Phương pháp 2 (One-Stage: Model tích hợp)	49
5.5.4	So sánh hiệu năng hai phương pháp	49
5.6	Đánh giá hệ thống phát hiện vi phạm	51

5.6.1	Phân tích kết quả nhận diện	51
5.6.2	Đánh giá về thách thức môi trường	52
5.6.3	Minh họa kết quả phát hiện vi phạm	52
5.7	Thách thức và Hạn chế	52
5.7.1	Các trường hợp khó với phát hiện phương tiện	52
5.7.2	Các trường hợp khó với nhận diện biển số	52
5.7.3	Hạn chế hiện tại của hệ thống	52
5.8	Tổng kết thực nghiệm	52
6	Kết luận và Hướng phát triển	56
6.1	Kết luận	56
6.1.1	Những đóng góp chính	56
6.1.2	Ý nghĩa thực tiễn	57
6.1.3	Những hạn chế	57
6.2	Hướng phát triển	57
6.2.1	Ngắn hạn (3-6 tháng)	57
6.2.2	Trung hạn (6-12 tháng)	58
6.2.3	Dài hạn (1-2 năm)	58
6.3	Lời kết	59

Danh sách hình vẽ

2.1	Cơ chế chia lưới và biểu diễn đầu ra của YOLO	7
2.2	Chu trình hoạt động của Bộ lọc Kalman	12
2.3	Mình họa quy trình liên kết hai giai đoạn của ByteTrack	15
2.4	So sánh góc chuyển động $\vec{v}$ với góc tham chiếu $\vec{r}$ (ngưỡng 50°). Vùng màu xanh lá: đi thẳng, vùng ngoài: rẽ trái/phải.	18
3.1	Pipeline tổng quan của hệ thống ALPR	19
3.2	Kiến trúc CRNN cho OCR trong PaddleOCR	20
4.1	Sơ đồ luồng xử lý chi tiết với cơ chế Direction Fusion và ALPR	28
4.2	Phân bố số lượng nhãn và vị trí không gian của các đối tượng	29
4.3	Pipeline xử lý nhận diện biển số xe	34
4.4	Pipeline Phương pháp 2: Phát hiện đồng thời xe và biển số trong một mô hình	36
5.1	Dữ liệu huấn luyện sau khi áp dụng Mosaic Augmentation	41
5.2	Đồ thị Loss và mAP trong quá trình huấn luyện phát hiện phương tiện	42
5.3	Ma trận nhầm lẫn chuẩn hóa (Normalized Confusion Matrix)	44
5.4	Kết quả dự đoán thực tế trên tập Validation (Batch 0)	45
5.5	Đường cong Loss và Metrics trong quá trình huấn luyện Plate Detection	46
5.6	Ma trận nhầm lẫn Plate Detection (Normalized)	47
5.7	Đường cong Loss và Metrics của mô hình tích hợp 6 lớp	48
5.8	Kết quả inference thực tế Phương pháp 1 (Two-Stage)	50
5.9	Mình họa các phương tiện với tín hiệu đèn giao thông	53
5.10	Mình họa xe vi phạm làn đường	54

Danh sách bảng

3.1	Các định dạng biển số xe Việt Nam	19
3.2	So sánh các phiên bản YOLOv8	22
3.3	Bảng hiệu chỉnh ký tự phổ biến	24
3.4	So sánh hai phương pháp triển khai ALPR	26
4.1	Tham số cấu hình huấn luyện YOLOv8 cho phát hiện phương tiện . .	30
4.2	Cấu hình huấn luyện Plate Detection	35
4.3	Bảng hiệu chỉnh ký tự	38
5.1	Cấu hình chi tiết môi trường thực nghiệm	40
5.2	Kết quả đánh giá hiệu năng mô hình YOLOv8n phát hiện phương tiện	43
5.3	Kết quả mAP@50 theo từng lớp phương tiện	43
5.4	Kết quả huấn luyện Plate Detection (Epoch 100)	47
5.5	Kết quả huấn luyện Mô hình tích hợp 6 lớp	47
5.6	Hiệu năng thực tế Phương pháp 1 (Two-Stage)	49
5.7	Hiệu năng thực tế Phương pháp 2 (One-Stage)	51
5.8	Bảng so sánh hiệu năng hai phương pháp phát hiện biển số	51
5.9	Tổng hợp kết quả thực nghiệm toàn hệ thống	55

Danh sách các từ viết tắt

AI	Artificial Intelligence (Trí tuệ nhân tạo)
ALPR	Automatic License Plate Recognition (Nhận dạng biển số tự động)
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
CPU	Central Processing Unit (Bộ xử lý trung tâm)
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
CIoU	Complete Intersection over Union (Hàm mất mát định vị)
DFL	Distribution Focal Loss
EMA	Exponential Moving Average
FN	False Negative (Âm tính giả - Bỏ sót đối tượng)
FP	False Positive (Dương tính giả - Bắt nhầm đối tượng)
FPS	Frames Per Second (Số khung hình trên giây)
GPU	Graphics Processing Unit (Bộ xử lý đồ họa)
HSV	Hue, Saturation, Value (Không gian màu)
IoU	Intersection over Union (Chỉ số Giao trên Hợp)
LSTM	Long Short-Term Memory
mAP	mean Average Precision (Độ chính xác trung bình)
MOT	Multi-Object Tracking (Theo dõi đa đối tượng)
OCR	Optical Character Recognition (Nhận dạng ký tự quang học)
R-CNN	Region-based Convolutional Neural Network
RGB	Red, Green, Blue (Không gian màu đỏ-lục-lam)
ROI	Region of Interest (Vùng quan tâm)

TP	True Positive (Dương tính thật - Dự đoán đúng)
VFL	Varifocal Loss
VNLP	Vietnamese License Plate (Biển số xe Việt Nam)
YOLO	You Only Look Once (Mô hình phát hiện đối tượng)

Chương 1

Mở đầu

1.1 Đặt vấn đề

1.1.1 Bối cảnh giao thông Việt Nam năm 2025

Bước sang năm 2025, quá trình đô thị hóa và chuyển đổi số tại Việt Nam đang diễn ra mạnh mẽ hơn bao giờ hết. Tại các thành phố lớn như Hà Nội và TP. Hồ Chí Minh, chính quyền đã bắt đầu triển khai thí điểm các hệ thống giao thông thông minh (Intelligent Transportation System - ITS) và camera phạt nguội tích hợp trí tuệ nhân tạo tại các nút giao trọng điểm. Những nỗ lực này bước đầu đã mang lại hiệu quả trong việc giảm ã và nâng cao ý thức người tham gia giao thông.

Tuy nhiên, việc áp dụng đại trà các hệ thống này vẫn gặp nhiều thách thức lớn:

- **Chi phí đầu tư cao:** Các hệ thống AI nhập ngoại hoặc các giải pháp phần cứng chuyên dụng đòi hỏi ngân sách lớn để triển khai và vận hành.
- **Đặc thù giao thông Việt Nam:** Mật độ xe máy dày đặc, hiện tượng giao thông hỗn hợp (mixed traffic) và hành vi di chuyển “điền vào chỗ trống” khiến các mô hình AI chuẩn quốc tế thường xuyên gặp sai sót hoặc bỏ lọt vi phạm.
- **Nhu cầu làm chủ công nghệ:** Việc tùy biến và tích hợp sâu vào hạ tầng sẵn có đòi hỏi phải làm chủ được công nghệ lõi.

1.1.2 Bài toán nhận diện biển số xe

Song song với giám sát vi phạm, nhận diện biển số xe tự động (Automatic License Plate Recognition - ALPR) là một trong những ứng dụng quan trọng của thị giác máy tính trong lĩnh vực giao thông thông minh. Tại Việt Nam, nhu cầu về hệ thống ALPR ngày càng tăng cao trong các ứng dụng thực tế như:

- Hệ thống thu phí tự động (ETC - Electronic Toll Collection)
- Quản lý bãi đỗ xe thông minh
- Giám sát giao thông và ghi nhận vi phạm
- Kiểm soát an ninh tại các cổng ra vào

Biển số xe Việt Nam có định dạng đặc thù với các ký tự chữ và số, được phân chia theo tỉnh/thành phố và loại phương tiện. Điều này đặt ra thách thức riêng cho các hệ thống ALPR quốc tế khi áp dụng vào thực tế Việt Nam do:

- Định dạng biển số khác biệt so với tiêu chuẩn quốc tế

- Chất lượng ảnh từ camera giám sát không đồng đều
- Điều kiện thời tiết và ánh sáng đa dạng
- Font chữ và kích thước biển số đặc thù

1.1.3 Định hướng nghiên cứu

Xuất phát từ thực tế trên, nhóm nghiên cứu lựa chọn đề tài **“Hệ thống giám sát giao thông thông minh và nhận diện biển số xe Việt Nam”** với mục tiêu xây dựng một hệ thống hoàn chỉnh có khả năng:

1. Chạy trên phần cứng phổ thông với tốc độ xử lý thời gian thực
2. Đảm bảo độ chính xác cao trong việc nhận diện các loại phương tiện đặc thù
3. Tự động phát hiện các hành vi vi phạm giao thông phức tạp
4. Nhận diện và đọc được biển số xe Việt Nam

Hệ thống sử dụng mô hình YOLOv8 tiên tiến, được tinh chỉnh (fine-tune) trên dữ liệu giao thông nội địa, kết hợp với các thuật toán hậu xử lý thông minh và công nghệ OCR hiện đại để giải quyết bài toán giám sát giao thông với chi phí thấp và hiệu quả cao.

1.2 Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống giám sát giao thông hoàn chỉnh với các khả năng sau:

1.2.1 Module phát hiện và theo dõi phương tiện

- **Nhận diện đa lớp phương tiện:** Phân loại chính xác 5 loại phương tiện phổ biến trong dòng giao thông hỗn hợp: Xe máy, Xe đạp, Ô tô con, Xe buýt và Xe tải.
- **Theo dõi và đếm lưu lượng:** Sử dụng YOLOv8 với thuật toán tracking ByteTrack tích hợp để theo dõi quỹ đạo di chuyển, đếm lưu lượng xe theo từng loại và từng làn đường.

1.2.2 Module phát hiện vi phạm

- **Giám sát tín hiệu giao thông:** Tự động định vị và nhận diện trạng thái màu sắc của đèn giao thông (Xanh/Đỏ/Vàng) bằng kỹ thuật xử lý ảnh trên không gian màu HSV.
- **Phát hiện vi phạm tự động:** Kết hợp logic máy tính để phát hiện chính xác các hành vi: Vượt đèn đỏ (dựa trên phân tích hướng di chuyển) và Đi sai làn đường quy định.

1.2.3 Module nhận diện biển số xe

- **Phát hiện biển số (Plate Detection):** Huấn luyện mô hình YOLOv8 để phát hiện vị trí biển số trong ảnh/video với độ chính xác cao (mAP@50 > 99%).
- **Nhận dạng ký tự (OCR):** Sử dụng PaddleOCR kết hợp các kỹ thuật tiền xử lý (CLAHE, Sharpening) để đọc chính xác các ký tự trên biển số.

- **Cơ chế ổn định kết quả:** Xây dựng Validation & Retry mechanism để đảm bảo kết quả OCR cuối cùng chính xác và đúng format biển số Việt Nam.

1.3 Phạm vi nghiên cứu

1.3.1 Dữ liệu

- **Dữ liệu phương tiện:** Sử dụng bộ dữ liệu thực tế tự quay chụp và thu thập để đảm bảo mô hình học được các đặc trưng của phương tiện tại Việt Nam (hơn 16.000 ảnh với 65.000 bounding box).
- **Dữ liệu biển số:** Sử dụng bộ dữ liệu VNLP (Vietnamese License Plate) để huấn luyện mô hình phát hiện biển số.

1.3.2 Công nghệ sử dụng

- **Phát hiện đối tượng:** YOLOv8n (phiên bản nano) – nhanh và nhẹ
- **Theo dõi đa đối tượng:** ByteTrack (tích hợp trong YOLOv8)
- **Xử lý tín hiệu đèn:** Thuật toán trên không gian màu HSV
- **Phát hiện vi phạm:** Thuật toán hình học với Direction Fusion
- **OCR biển số:** PaddleOCR với kiến trúc CRNN + CTC
- **Tiền xử lý ảnh:** CLAHE và Sharpening

1.3.3 Giới hạn

- **Loại camera:** Hệ thống tập trung xử lý video từ camera giám sát cố định (CCTV) với góc quay từ trên cao.
- **Điều kiện ánh sáng:** Hoạt động tốt trong điều kiện ánh sáng ban ngày và ban đêm có đèn đường.
- **Loại biển số:** Hỗ trợ biển số xe Việt Nam cho cả ô tô (7-8 ký tự) và xe máy (8-9 ký tự), tập trung vào biển trắng 1 dòng.
- **Đầu vào:** Video giao thông từ camera cố định.
- **Đầu ra:** Thông tin phương tiện, cảnh báo vi phạm và chuỗi ký tự biển số đã được nhận dạng.

1.4 Cấu trúc báo cáo

Báo cáo được trình bày với bố cục như sau:

- **Chương 1: Mở đầu.** Trình bày bối cảnh giao thông Việt Nam, lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu.
- **Chương 2: Cơ sở lý thuyết – Phát hiện và theo dõi đối tượng.** Trình bày nền tảng toán học của mô hình YOLOv8, cơ chế tracking tích hợp (ByteTrack) và các kỹ thuật xử lý ảnh trong nhận diện tín hiệu đèn và hình học tính toán.
- **Chương 3: Cơ sở lý thuyết – Nhận diện biển số xe.** Trình bày kiến trúc PaddleOCR, các kỹ thuật tiền xử lý ảnh (CLAHE, Sharpening) và cấu trúc biển số xe Việt Nam.
- **Chương 4: Phương pháp thực hiện.** Mô tả chi tiết kiến trúc hệ thống,

quy trình xử lý dữ liệu, huấn luyện mô hình, các thuật toán xác định hướng di chuyển, phát hiện vi phạm và pipeline nhận diện biển số.

- **Chương 5: Thực nghiệm và Đánh giá.** Trình bày môi trường thực nghiệm, kết quả huấn luyện, kết quả chạy thực tế trên video và phân tích hiệu năng hệ thống.
- **Chương 6: Kết luận và Hướng phát triển.** Tổng kết các đóng góp của đề tài, những hạn chế còn tồn tại và định hướng nâng cấp trong tương lai.

Chương 2

Cơ sở lý thuyết: Phát hiện và theo dõi đối tượng

2.1 Tổng quan về bài toán Phát hiện đối tượng

2.1.1 Giới thiệu chung

Phát hiện đối tượng (Object Detection) là một bài toán nền tảng và thiết yếu trong lĩnh vực Thị giác máy tính (Computer Vision). Mục đích chính của bài toán là xác định sự hiện diện của các thực thể quan tâm trong một hình ảnh kỹ thuật số hoặc video, đồng thời định vị chính xác vị trí và gán nhãn phân loại cho từng thực thể đó.

Khác với bài toán phân loại ảnh (Image Classification) đơn thuần chỉ gán một nhãn duy nhất cho toàn bộ bức ảnh, phát hiện đối tượng giải quyết đồng thời hai nhiệm vụ phức tạp:

1. **Định vị (Localization):** Xác định vị trí của đối tượng thông qua một khung bao (Bounding Box).
2. **Phân loại (Classification):** Xác định danh mục (class) của đối tượng nằm trong khung bao đó.

Bài toán này đóng vai trò tiền đề cho nhiều ứng dụng thực tế quan trọng như: xe tự hành (autonomous driving), hệ thống giám sát an ninh, chẩn đoán y tế qua hình ảnh và tương tác người-máy.

2.1.2 Định nghĩa toán học

Về mặt hình thức, xét một ảnh đầu vào $I \in \mathbb{R}^{H \times W \times C}$, trong đó H, W, C lần lượt là chiều cao, chiều rộng và số kênh màu của ảnh. Mục tiêu của mô hình $f(I)$ là dự đoán một tập hợp các bounding box $B = \{b_1, b_2, \dots, b_N\}$, trong đó mỗi b_i đại diện cho một đối tượng được phát hiện và được định nghĩa bởi vector:

$$b_i = (x_i, y_i, w_i, h_i, c_i, p_i) \quad (2.1)$$

Các tham số được giải thích cụ thể như sau:

- (x_i, y_i) : Tọa độ tâm của bounding box, thường được chuẩn hóa theo kích thước ảnh hoặc so với lưới đặc trưng (feature grid).

- (w_i, h_i) : Chiều rộng và chiều cao của bounding box. Trong các mô hình hiện đại, giá trị này thường được dự đoán dưới dạng độ lệch (offset) log-space so với các khung neo (anchor) định trước.
- $c_i \in \{1, \dots, K\}$: Nhãn lớp của đối tượng, thuộc một trong K danh mục quan tâm (ví dụ: người, xe, chó, mèo).
- $p_i \in [0, 1]$: Độ tin cậy (confidence score), phản ánh xác suất mà mô hình tin rằng bounding box này chứa một đối tượng thực sự thuộc lớp c_i .

2.1.3 Thách thức của bài toán

Việc ánh xạ từ không gian pixel sang không gian ngữ nghĩa của các bounding box gặp phải nhiều thách thức lớn, bao gồm:

- **Sự biến thiên về tỷ lệ (Scale Variation)**: Các đối tượng trong ảnh có thể xuất hiện với kích thước rất khác nhau, từ vài pixel đến chiếm toàn bộ khung hình.
- **Sự che khuất (Occlusion)**: Các đối tượng có thể bị che khuất một phần bởi các đối tượng khác hoặc nền, làm mất đi các đặc trưng quan trọng để nhận dạng.
- **Điều kiện ánh sáng**: Sự thay đổi về ánh sáng, bóng đổ, phản chiếu ảnh hưởng đến chất lượng ảnh đầu vào.
- **Giao thông hỗn hợp**: Đặc thù tại Việt Nam với nhiều loại phương tiện di chuyển xen kẽ nhau.

2.2 Cơ sở lý thuyết về mô hình YOLO

2.2.1 Hợp nhất bài toán thành một mạng duy nhất

Trước khi kiến trúc YOLO (You Only Look Once) ra đời, các hệ thống phát hiện đối tượng kinh điển thường xử lý bài toán theo quy trình hai giai đoạn tách biệt (Two-Stage Detectors). Các phương pháp như R-CNN sử dụng thuật toán đề xuất vùng (Region Proposal) như Selective Search để tìm kiếm các vùng ứng viên, sau đó mới áp dụng bộ phân loại trên từng vùng này. Tuy đạt độ chính xác cao, nhưng quy trình này tốn kém về mặt tính toán và khó tối ưu hóa toàn cục.

YOLO, được giới thiệu bởi Redmon et al. (2015), đã thay đổi hoàn toàn tư duy này bằng cách tái định nghĩa việc phát hiện đối tượng thành một bài toán **hồi quy duy nhất** (Single Regression Problem). Thay vì quét ảnh nhiều lần, YOLO chỉ cần một lần truyền thẳng (forward pass) qua mạng nơ-ron tích chập (CNN) để dự đoán đồng thời cả vị trí khung bao (bounding box) và xác suất lớp đối tượng. Điều này cho phép mô hình nhìn thấy ngữ cảnh toàn cục của bức ảnh, giảm thiểu các sai sót do nhầm lẫn hậu cảnh (background).

2.2.2 Cơ chế chia lưới và Biểu diễn đầu ra

Cơ chế hoạt động cốt lõi của mọi phiên bản YOLO dựa trên việc chia ảnh đầu vào thành một lưới kích thước $S \times S$. Quy tắc gán nhãn được định nghĩa như sau:

- Nếu tâm (center) của một đối tượng thực tế rơi vào ô lưới nào, ô lưới đó chịu trách nhiệm phát hiện đối tượng đó.

- Mỗi ô lưới sẽ dự đoán B khung bao dự kiến, cùng với độ tin cậy (confidence) cho mỗi khung bao.

Về mặt toán học, độ tin cậy C_{pred} được định nghĩa là tích của xác suất có đối tượng và độ chồng lấn (IoU) với thực tế:

$$C_{pred} = Pr(\text{Object}) \times \text{IoU}_{pred}^{truth} \quad (2.2)$$

Nếu không có đối tượng trong ô lưới, $Pr(\text{Object}) = 0$, dẫn đến $C_{pred} = 0$.

Đầu ra của mạng là một tensor 3 chiều có kích thước $S \times S \times (B \times 5 + K)$, trong đó K là số lớp đối tượng. Mỗi vector dự đoán bao gồm 5 tham số cơ bản:

$$\hat{y} = (t_x, t_y, t_w, t_h, t_o) \quad (2.3)$$

Trong đó (t_x, t_y) là tọa độ tâm cục bộ trong ô lưới, (t_w, t_h) là kích thước khung bao được chuẩn hóa, và t_o là độ tin cậy.

../BỔ_ÁO_CỔ_ÁO_ADVC/Figures/chia_luoi.png

Hình 2.1: Cơ chế chia lưới và biểu diễn đầu ra của YOLO

2.2.3 Kiến trúc mạng tổng quát

Hầu hết các phiên bản YOLO hiện đại đều tuân theo kiến trúc Meta-architecture gồm 3 thành phần chính:

1. **Backbone (Xương sống):** Là một mạng CNN đóng vai trò trích xuất đặc trưng từ ảnh đầu vào ở các mức độ phân giải khác nhau.

2. **Neck (Cổ):** Sử dụng các kỹ thuật như Feature Pyramid Network (FPN) hoặc Path Aggregation Network (PANet) để trộn và kết hợp các đặc trưng từ Backbone, giúp mô hình nhận diện tốt cả đối tượng nhỏ và lớn.
3. **Head (Đầu):** Thực hiện nhiệm vụ cuối cùng là dự đoán lớp và tọa độ khung bao trên các bản đồ đặc trưng đã được tổng hợp.

2.2.4 Sự tiến hóa và Lựa chọn YOLOv8

Trải qua quá trình phát triển, YOLO đã có những bước tiến vượt bậc:

- **YOLOv1-v3:** Tập trung vào tốc độ thời gian thực, giới thiệu khái niệm Anchor Box để ổn định việc huấn luyện.
- **YOLOv4-v5:** Tối ưu hóa mạnh mẽ về kỹ thuật huấn luyện (Training Tricks) như Mosaic Augmentation và cải tiến kiến trúc với CSPNet để tăng cường luồng gradient.
- **YOLOv6-v8 (2022-2023):** Đánh dấu sự chuyển dịch sang kiến trúc **Anchor-free** (không dùng khung neo) và **Decoupled Head** (tách riêng nhánh phân loại và hồi quy).

Lý do lựa chọn YOLOv8: Đề tài này lựa chọn **YOLOv8** (Ultralytics, 2023) làm mô hình chủ đạo. Đây là phiên bản không quá nặng, kế thừa tinh hoa của các thế hệ trước nhưng sở hữu các cải tiến vượt trội phù hợp cho bài toán giao thông tại Việt Nam:

- **Anchor-free:** Loại bỏ nhu cầu thiết kế kích thước anchor box thủ công, giúp mô hình linh hoạt hơn với các đối tượng có tỷ lệ kích thước biến thiên lớn (như xe máy chở hàng công kênh, xe buýt).
- **C2f Module:** Thay thế module C3 cũ, giúp mô hình nhẹ hơn nhưng trích xuất đặc trưng phong phú hơn nhờ các luồng gradient song song.
- **Task Aligned Assigner:** Cơ chế gán nhãn động thông minh, giúp cân bằng tốt hơn giữa mẫu dương và mẫu âm trong quá trình huấn luyện.

2.3 Mô hình YOLOv8 chi tiết

YOLOv8 là mô hình phát hiện đối tượng một giai đoạn (one-stage detector) tiên tiến, coi bài toán phát hiện đối tượng là một bài toán hồi quy trực tiếp từ các pixel của ảnh sang tọa độ bounding box và xác suất lớp.

2.3.1 Cơ chế Anchor-free

Khác với các thế hệ trước dựa trên Anchor Box (khung neo), YOLOv8 áp dụng phương pháp **Anchor-free**. Thay vì dự đoán độ lệch (offset) so với anchor box, mô hình dự đoán trực tiếp khoảng cách từ tâm của ô lưới (grid cell) đến 4 cạnh của bounding box thực tế.

Giả sử tâm của một đối tượng rơi vào ô lưới tại tọa độ (c_x, c_y) , mô hình sẽ dự đoán vector 4 chiều $t = (l, t, r, b)$ đại diện cho khoảng cách tới cạnh trái, trên, phải, và dưới.

2.3.2 Các thành phần kiến trúc đặc trưng

Module C2f (Cross Stage Partial with 2 flow)

YOLOv8 giới thiệu module C2f để thay thế cho module C3 trong các phiên bản trước. C2f được thiết kế dựa trên ý tưởng của kiến trúc ELAN (Efficient Layer Aggregation Network), giúp cải thiện luồng gradient thông tin trong mạng nơ-ron mà không làm tăng đáng kể chi phí tính toán.

Cơ chế hoạt động của C2f bao gồm hai bước:

1. Tách tensor đầu vào thành hai phần theo kênh (channel-wise split).
 2. Một phần đi qua chuỗi các lớp Bottleneck, phần còn lại đi thẳng (shortcut).
- Cuối cùng, tất cả các nhánh được nối (concatenate) lại với nhau.

Công thức tổng quát cho đầu ra Y của khối C2f có thể mô tả như sau:

$$Y = \text{Conv}_{1 \times 1}([X_1, \text{Block}_1(X_2), \text{Block}_2(\text{Block}_1(X_2)), \dots]) \quad (2.4)$$

Kiến trúc này giúp mô hình học được các đặc trưng phong phú hơn (richer features) nhờ việc kết hợp thông tin ở nhiều độ sâu khác nhau.

Module SPPF (Spatial Pyramid Pooling Fast)

Nằm ở cuối phần Backbone, module SPPF có nhiệm vụ trích xuất các đặc trưng ngữ cảnh không gian (spatial context features) ở nhiều tỷ lệ khác nhau. Thay vì sử dụng các bộ lọc có kích thước lớn gây tốn kém tính toán, SPPF thực hiện tuần tự 3 lần phép gộp cực đại (Max Pooling) với kích thước 5×5 .

Đầu ra là sự kết hợp (concatenation) của đầu vào gốc và kết quả của 3 lần pooling:

$$\text{Out} = \text{Concat}(X, \text{MaxPool}(X), \text{MaxPool}^2(X), \text{MaxPool}^3(X)) \quad (2.5)$$

Điều này giúp mô hình nhận diện tốt các đối tượng có kích thước thay đổi lớn (như xe buýt so với xe máy) mà vẫn đảm bảo tốc độ thực thi nhanh.

Chiến lược gán nhãn Task Aligned Assigner

Một trong những cải tiến quan trọng nhất của YOLOv8 là loại bỏ cơ chế gán nhãn dựa trên IoU tĩnh (static IoU-based assignment) và thay thế bằng **Task Aligned Assigner**. Chiến lược này chọn lọc các mẫu dương (positive samples) dựa trên sự cân bằng giữa điểm phân loại và độ chính xác định vị.

Điểm số căn chỉnh (alignment score) t cho mỗi anchor point được tính như sau:

$$t = s^\alpha \times u^\beta \quad (2.6)$$

Trong đó:

- s : Điểm số phân loại dự đoán (predicted classification score).
- u : Chỉ số IoU giữa box dự đoán và box thực tế.
- α và β : Các tham số điều trọng số (thường $\alpha = 0.5, \beta = 6.0$).

Chỉ những mẫu có điểm t cao nhất mới được chọn làm mẫu dương để huấn luyện. Cơ chế này giúp mô hình tập trung vào các dự đoán chất lượng cao, nơi mà "phân loại đúng" đi đôi với "định vị chính xác".

2.3.3 Hàm mất mát (Loss Function)

Quá trình huấn luyện YOLOv8 nhằm tối thiểu hóa hàm mất mát tổng hợp L_{total} , bao gồm hai thành phần chính: mất mát về định vị (L_{box}) và mất mát về phân loại (L_{cls}).

$$L_{total} = \lambda_{box}L_{box} + \lambda_{cls}L_{cls} + \lambda_{dfl}L_{dfl} \quad (2.7)$$

Mất mát định vị (Box Loss)

YOLOv8 sử dụng hàm mất mát **CIoU (Complete IoU)** để đo lường sai số giữa khung dự đoán (B_{pred}) và khung thực tế (B_{gt}). CIoU khắc phục nhược điểm của IoU truyền thống bằng cách xem xét thêm khoảng cách tâm và tỷ lệ khung hình.

Công thức CIoU được định nghĩa như sau:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (2.8)$$

Trong đó:

- $IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|}$ là chỉ số giao trên hợp.
- $\rho(\cdot)$ là khoảng cách Euclidean giữa tâm của hai bounding box $\mathbf{b}$ và $\mathbf{b}^{gt}$.
- c là đường chéo của bao lồi nhỏ nhất chứa cả hai box.
- α là tham số cân bằng trọng số.
- v đo lường sự tương đồng về tỷ lệ khung hình (aspect ratio consistency):

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2 \quad (2.9)$$

Ngoài ra, YOLOv8 còn kết hợp **Distribution Focal Loss (DFL)** để tối ưu hóa phân phối xác suất của các cạnh bounding box, giúp mô hình hội tụ nhanh hơn khi dự đoán các box có biên mờ.

Mất mát phân loại (Classification Loss)

Đối với nhánh phân loại, YOLOv8 sử dụng **Varifocal Loss (VFL)** hoặc Binary Cross Entropy (BCE). VFL giúp cân bằng giữa các mẫu dương (positive samples) và mẫu âm (negative samples) khó học:

$$VFL(p, q) = \begin{cases} -q(q \log(p) + (1 - q) \log(1 - p)) & \text{nếu } q > 0 \\ -\alpha p^\gamma \log(1 - p) & \text{nếu } q = 0 \end{cases} \quad (2.10)$$

Trong đó p là xác suất dự đoán và q là điểm IoU mục tiêu (target IoU score).

2.4 Các độ đo đánh giá (Evaluation Metrics)

Để đánh giá hiệu năng mô hình, chúng tôi sử dụng độ đo mAP (mean Average Precision) tiêu chuẩn.

2.4.1 Precision và Recall

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2.11)$$

Trong đó: TP (True Positive) là số detection đúng ($IoU > \text{ngưỡng}$), FP (False Positive) là detection sai, FN (False Negative) là đối tượng bị bỏ sót.

2.4.2 Average Precision (AP)

AP là diện tích dưới đường cong Precision-Recall (PR Curve) cho một lớp đối tượng cụ thể, được tính bằng tích phân:

$$AP = \int_0^1 p(r)dr \quad (2.12)$$

Trong thực tế, AP được tính bằng cách nội suy tại 101 điểm recall.

2.4.3 Mean Average Precision (mAP)

mAP là trung bình cộng của AP trên tất cả N lớp đối tượng (trong đề tài này $N = 5$: xe máy, ô tô, xe tải, xe buýt, xe đạp):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2.13)$$

Chỉ số **mAP@50** (tính tại ngưỡng $IoU=0.5$) và **mAP@50:95** (trung bình trên các ngưỡng IoU từ 0.5 đến 0.95) được sử dụng làm tiêu chí chính để so sánh các mô hình trong báo cáo này.

2.5 Theo dõi đa đối tượng (Multi-Object Tracking)

Bài toán Theo dõi đa đối tượng (Multi-Object Tracking - MOT) nhằm mục đích ước lượng quỹ đạo của các đối tượng trong video. Giả sử tại mỗi khung hình t , ta nhận được một tập hợp các detection $D_t = \{d_1, d_2, \dots, d_n\}$. Mục tiêu là gán một ID duy nhất k cho mỗi chuỗi detection $(d_{t_1}^k, d_{t_2}^k, \dots)$ tương ứng với cùng một thực thể thực tế.

Hệ thống sử dụng phương pháp **Tracking-by-Detection** kết hợp với thuật toán **ByteTrack**, trong đó thành phần cốt lõi là Bộ lọc Kalman để ước lượng trạng thái và thuật toán Hungarian để liên kết dữ liệu.

2.5.1 Bộ lọc Kalman (Kalman Filter)

Bộ lọc Kalman là một thuật toán đệ quy tối ưu để ước lượng trạng thái của một hệ thống động lực học từ các phép đo nhiễu. Trong bài toán tracking phương tiện, chúng tôi mô hình hóa chuyển động của xe là chuyển động đều (constant velocity model).

Không gian trạng thái (State Space)

Trạng thái của một phương tiện tại thời điểm k được biểu diễn bởi vector 8 chiều $\mathbf{x}_k$:

$$\mathbf{x}_k = [u_k, v_k, a_k, h_k, \dot{u}_k, \dot{v}_k, \dot{a}_k, \dot{h}_k]^T \quad (2.14)$$

Trong đó:

- (u_k, v_k) : Tọa độ tâm của bounding box.

- a_k : Tỷ lệ khung hình (aspect ratio w/h).
- h_k : Chiều cao của bounding box.
- $\dot{u}_k, \dot{v}_k, \dot{a}_k, \dot{h}_k$: Vận tốc biến thiên tương ứng của các tham số trên.

Quá trình Dự đoán (Prediction)

Dựa trên trạng thái ở thời điểm trước đó $k-1$, bộ lọc dự đoán trạng thái tiên nghiệm (a priori) tại thời điểm k :

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} \quad (2.15)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (2.16)$$

Trong đó:

- $\mathbf{F}_k$: Ma trận chuyển trạng thái (state transition matrix), mô tả mô hình vật lý của chuyển động.
- $\mathbf{P}_{k|k-1}$: Ma trận hiệp phương sai số dự đoán.
- $\mathbf{Q}_k$: Ma trận hiệp phương sai nhiễu của quá trình (process noise).

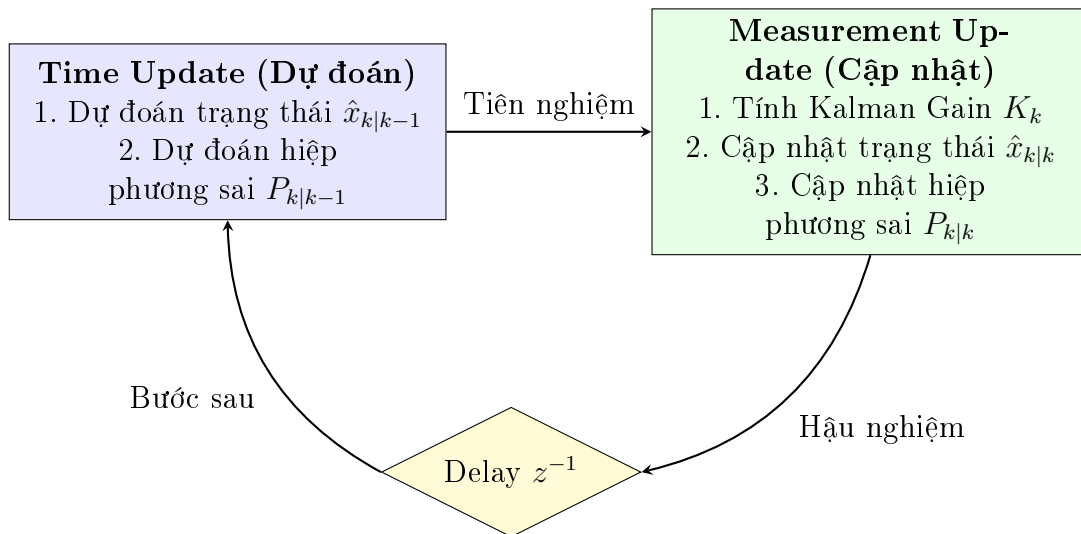
Quá trình Cập nhật (Update)

Khi nhận được phép đo mới (measurement) $\mathbf{z}_k$ từ YOLOv8 (tọa độ bounding box $[u, v, a, h]^T$), bộ lọc sẽ hiệu chỉnh trạng thái dự đoán để có trạng thái hậu nghiệm (a posteriori) chính xác hơn:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (2.17)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1}) \quad (2.18)$$

Trong đó $\mathbf{K}_k$ là độ lợi Kalman (Kalman Gain), $\mathbf{H}_k$ là ma trận quan sát, và $\mathbf{R}_k$ là nhiễu đo lường.



Hình 2.2: Chu trình hoạt động của Bộ lọc Kalman

2.5.2 Liên kết dữ liệu (Data Association)

Liên kết dữ liệu là bước quyết định trong hệ thống theo dõi, đóng vai trò cầu nối giữa các kết quả phát hiện rời rạc (discrete detections) và các quỹ đạo liên tục (continuous trajectories). Giả sử tại thời điểm t , ta có tập hợp M quỹ đạo hiện hành $T = \{t_1, t_2, \dots, t_M\}$ và tập hợp N đối tượng vừa được phát hiện $D = \{d_1, d_2, \dots, d_N\}$.

Bài toán đặt ra là tìm một ánh xạ ghép cặp 1-1 tối ưu giữa T và D sao cho tổng độ sai lệch (chi phí) là nhỏ nhất. Vấn đề này được mô hình hóa dưới dạng bài toán **Phân công Tuyến tính (Linear Assignment Problem)** trên đồ thị hai phía (bipartite graph).

Hàm mục tiêu cần tối thiểu hóa được biểu diễn như sau:

$$\min \sum_{i=1}^M \sum_{j=1}^N C_{i,j} X_{i,j} \quad (2.19)$$

Thỏa mãn các điều kiện ràng buộc:

$$\begin{cases} \sum_{j=1}^N X_{i,j} \leq 1, & \forall i = 1 \dots M \\ \sum_{i=1}^M X_{i,j} \leq 1, & \forall j = 1 \dots N \\ X_{i,j} \in \{0, 1\} \end{cases} \quad (2.20)$$

Trong đó:

- $X_{i,j}$ là biến quyết định nhị phân: $X_{i,j} = 1$ nếu quỹ đạo t_i được ghép với detection d_j , và $X_{i,j} = 0$ nếu ngược lại.
- Các điều kiện ràng buộc đảm bảo nguyên tắc "độc nhất": mỗi quỹ đạo chỉ được cập nhật bởi tối đa một detection, và mỗi detection chỉ được gán cho tối đa một quỹ đạo.
- $C_{i,j}$ là ma trận chi phí (cost matrix), biểu thị mức độ "không tương đồng" giữa t_i và d_j .

Trong hệ thống sử dụng ByteTrack, chúng tôi sử dụng độ đo không gian IoU (Intersection over Union) để xây dựng ma trận chi phí thay vì dùng khoảng cách Euclid hay Mahalanobis, nhằm tận dụng tối đa khả năng định vị chính xác của YOLOv8:

$$C_{i,j} = 1 - \text{IoU}(\text{Box}(t_i), \text{Box}(d_j)) \quad (2.21)$$

Giá trị $C_{i,j}$ càng nhỏ đồng nghĩa với việc vùng bao dự đoán của track và vùng bao phát hiện thực tế chồng lấn càng nhiều, khả năng chúng là cùng một đối tượng càng cao.

Để giải quyết bài toán tối ưu tổ hợp này, chúng tôi sử dụng **thuật toán Hungarian** (hay còn gọi là thuật toán Kuhn-Munkres). Thuật toán này đảm bảo tìm ra nghiệm tối ưu toàn cục với độ phức tạp thời gian là $O(n^3)$, hoàn toàn đáp ứng được yêu cầu xử lý thời gian thực của hệ thống.

2.5.3 Thuật toán ByteTrack

Trong các phương pháp theo dõi đa đối tượng truyền thống (như DeepSORT hay SORT), một ngưỡng tin cậy (confidence threshold) cao thường được áp dụng để

lọc bỏ các nhiễu nền (background noise). Ví dụ, chỉ những bounding box có điểm tin cậy > 0.5 mới được giữ lại để thực hiện liên kết. Tuy nhiên, trong bối cảnh giao thông thực tế, các đối tượng xe máy hoặc ô tô khi bị che khuất một phần (partial occlusion) hoặc bị mờ do chuyển động nhanh (motion blur) thường có điểm tin cậy giảm xuống thấp (ví dụ: $0.1 - 0.4$). Việc loại bỏ thẳng tay các detection này dẫn đến hệ quả là các quỹ đạo (tracks) bị đứt gãy, gây ra hiện tượng chuyển đổi ID (ID Switch) sai lệch.

Để giải quyết vấn đề này, chúng tôi sử dụng ByteTrack (Zhang et al., 2022). Đây là một thuật toán tracking đơn giản nhưng hiệu quả vượt trội, dựa trên tư tưởng: *"Tận dụng mọi detection, kể cả những detection có điểm tin cậy thấp, thay vì loại bỏ chúng"*.

Quy trình liên kết dữ liệu (data association) của ByteTrack được thực hiện qua cơ chế phân tầng (hierarchical matching) gồm các bước chi tiết sau:

Phân loại Detection

Tại mỗi khung hình t , tập hợp các detection thu được từ YOLOv8 được chia làm hai tập con dựa trên ngưỡng phân loại τ (thường là 0.5):

- $\mathcal{D}_{high}$: Tập các detection có độ tin cậy cao ($score > \tau$).
- $\mathcal{D}_{low}$: Tập các detection có độ tin cậy thấp ($\epsilon < score \leq \tau$), trong đó ϵ là ngưỡng tối thiểu (ví dụ 0.1) để loại bỏ hoàn toàn nhiễu.

Quy trình liên kết hai giai đoạn

Giai đoạn 1 (High Score Matching):

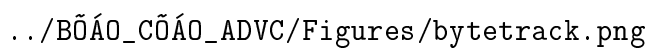
Hệ thống ưu tiên liên kết các detection chất lượng cao ($\mathcal{D}_{high}$) với các quỹ đạo hiện có (existing tracks) thông qua bộ lọc Kalman.

- Sử dụng bộ lọc Kalman để dự đoán vị trí của các Track ở khung hình hiện tại.
- Tính toán ma trận độ tương đồng giữa $\mathcal{D}_{high}$ và các Track dựa trên chỉ số IoU (Intersection over Union).
- Sử dụng thuật toán Hungarian để ghép cặp tối ưu.
- *Kết quả*: Các detection được ghép thành công sẽ cập nhật trạng thái cho Track tương ứng. Các detection chưa được ghép sẽ được giữ lại để khởi tạo Track mới. Các Track chưa tìm thấy đối tượng ghép (Unmatched Tracks) sẽ được chuyển sang giai đoạn 2.

Giai đoạn 2 (Low Score Matching):

Đây là bước đột phá của ByteTrack. Hệ thống cố gắng "cứu" các Track bị bỏ sót ở giai đoạn 1 bằng cách tìm kiếm trong tập detection độ tin cậy thấp ($\mathcal{D}_{low}$).

- Giả định rằng các Track chưa được ghép nối ở Giai đoạn 1 thường là do đối tượng bị che khuất, làm giảm confidence score của mô hình detection.
- Thực hiện ghép nối giữa các Unmatched Tracks (từ Giai đoạn 1) với $\mathcal{D}_{low}$, cũng sử dụng IoU làm trọng số.
- *Kết quả*: Nếu ghép nối thành công, Track đó sẽ được duy trì (giữ nguyên ID cũ) dù điểm detection thấp. Điều này giúp quỹ đạo liền mạch ngay cả khi xe đi khuất sau xe khác.



`../BỔ_ÁO_CỔ_ÁO_ADVC/Figures/bytetrack.png`

Hình 2.3: Minh họa quy trình liên kết hai giai đoạn của ByteTrack

Quản lý vòng đời Track

- **Track Birth (Sinh):** Các detection trong $\mathcal{D}_{high}$ không ghép được với bất kỳ track nào ở Giai đoạn 1 sẽ được xem là đối tượng mới xuất hiện và được cấp ID mới.
- **Track Death (Hủy):** Các Track không ghép được với bất kỳ detection nào (cả high và low) trong K khung hình liên tiếp (ví dụ $K = 30$) sẽ bị coi là đã ra khỏi khung hình và bị xóa bỏ.

Ưu điểm đối với bài toán giao thông Việt Nam

Việc áp dụng ByteTrack mang lại hai lợi ích lớn cho đề tài:

1. **Xử lý che khuất:** Trong điều kiện giao thông hỗn hợp tại Việt Nam, xe máy thường xuyên bị xe buýt hoặc ô tô che khuất. ByteTrack giúp "gấp" lại các xe máy này ở giai đoạn 2, giảm thiểu việc đếm sai lưu lượng.
2. **Tốc độ thực thi cao:** ByteTrack chỉ sử dụng thông tin hình học (geometric information) để liên kết mà không cần trích xuất đặc trưng ngoại hình (appearance features) bằng mạng nơ-ron sâu như DeepSORT. Điều này giúp giảm tải tính toán cho GPU, đảm bảo hệ thống duy trì được FPS cao (frames per second).

2.6 Xử lý ảnh và Hình học tính toán

Để giải quyết các bài toán cụ thể như nhận diện trạng thái đèn giao thông và xác định hành vi phương tiện (lấn làn, vượt đèn), hệ thống kết hợp các kỹ thuật xử lý ảnh trên không gian màu và các thuật toán hình học tính toán.

2.6.1 Không gian màu HSV trong nhận diện tín hiệu đèn

Chúng tôi chuyển đổi ảnh từ không gian màu RGB sang HSV để tách biệt thông tin màu sắc (Hue) khỏi độ sáng (Value), giúp thuật toán bền vững với thay đổi ánh sáng môi trường.

Quá trình chuyển đổi từ một điểm ảnh (R, G, B) sang (H, S, V) được thực hiện như sau. Đầu tiên, chuẩn hóa các giá trị $R, G, B \in [0, 1]$. Đặt $C_{max} = \max(R, G, B)$, $C_{min} = \min(R, G, B)$ và $\Delta = C_{max} - C_{min}$.

- **Value (V):** Độ sáng của điểm ảnh.

$$V = C_{max} \quad (2.22)$$

- **Saturation (S):** Độ bão hòa màu.

$$S = \begin{cases} 0 & \text{nếu } C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{nếu } C_{max} \neq 0 \end{cases} \quad (2.23)$$

- **Hue (H):** Góc sắc độ (thể hiện màu sắc thuần túy).

$$H = \begin{cases} 60^\circ \times (\frac{G-B}{\Delta} \bmod 6) & \text{nếu } C_{max} = R \\ 60^\circ \times (\frac{B-R}{\Delta} + 2) & \text{nếu } C_{max} = G \\ 60^\circ \times (\frac{R-G}{\Delta} + 4) & \text{nếu } C_{max} = B \end{cases} \quad (2.24)$$

Hệ thống sử dụng các ngưỡng (threshold) trên kênh H và S để đếm số lượng pixel thuộc về màu Đỏ, Vàng, hoặc Xanh. Trạng thái đèn được xác định bởi màu có số lượng pixel vượt trội (dominant color).

2.6.2 Hình học tính toán trong phát hiện vi phạm

Kiểm tra xe trong làn đường

Làn đường là đa giác (không bắt buộc lồi) $P = \{v_1, \dots, v_n\}$. Tâm xe $C(x, y)$ được kiểm tra bằng hàm `cv2.pointPolygonTest` (tương đương ray casting): kết quả ≥ 0 nếu điểm nằm trong/biên, < 0 nếu ngoài.

Khoảng cách tới vạch dừng và sự kiện cắt qua

Vạch dừng: đoạn $P_1(x_1, y_1) - P_2(x_2, y_2)$. Khoảng cách vuông góc (đường thẳng qua P_1, P_2):

$$d(C, P_1P_2) = \frac{|(x_2 - x_1)(y_1 - y_C) - (x_1 - x_C)(y_2 - y_1)|}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}$$

Trong code, khoảng cách được "kẹp" vào đoạn (nếu hình chiếu ra ngoài đoạn, lấy khoảng cách tới đầu mút). Sự kiện vượt vạch khi: (i) $d < \delta$ (mặc định 15–20 px), (ii) track ID chưa nằm trong tập xe đã qua vạch.

Ước lượng hướng di chuyển dựa trên góc tham chiếu

Hướng di chuyển đi thẳng được xác định bởi góc tham chiếu θ_{ref} (đơn vị: độ). Giá trị này mặc định là 90° và được cập nhật khi hệ thống thiết lập Reference Vector.

Quá trình ước lượng dựa trên lịch sử vị trí bao gồm tối đa 10 điểm dữ liệu gần nhất. Để đảm bảo độ tin cậy, hệ thống yêu cầu tối thiểu 5 điểm dữ liệu và khoảng cách dịch chuyển phải lớn hơn 30 pixel. Từ vector chuyển động $\vec{v} = (dx, dy)$, góc di chuyển tuyệt đối θ và góc lệch tương đối θ_{rel} được tính toán như sau:

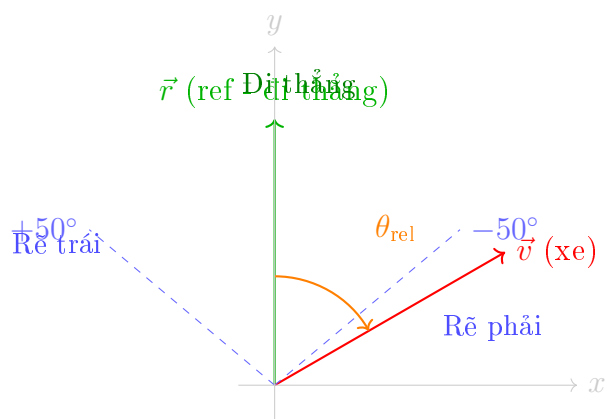
$$\theta = \text{atan2}(dy, dx) \quad (\text{độ})$$

$$\theta_{\text{rel}} = \theta - \theta_{\text{ref}}, \quad \text{với } \theta_{\text{rel}} \in [-180^\circ, 180^\circ]$$

Phân loại hướng di chuyển với ngưỡng 50° :

$$\begin{cases} \text{Đi thẳng} & |\theta_{\text{rel}}| \leq 50^\circ \\ \text{Rẽ phải} & \theta_{\text{rel}} < -50^\circ \\ \text{Rẽ trái} & \theta_{\text{rel}} > 50^\circ \end{cases}$$

Giải thích lựa chọn ngưỡng 50° : Ngưỡng 50° được chọn dựa trên thực nghiệm với các video giao thông tại Việt Nam, phù hợp với đặc thù giao thông hỗn hợp nơi xe thường di chuyển theo nhiều hướng khác nhau tại các ngã tư. Ngưỡng này đủ rộng để không nhận diện sai các xe đi thẳng nhưng có độ lệch nhỏ do điều kiện đường sá, đồng thời đủ chặt để phân biệt rõ ràng giữa đi thẳng và rẽ.



Hình 2.4: So sánh góc chuyển động $\vec{v}$ với góc tham chiếu $\vec{r}$ (ngưỡng 50°). Vùng màu xanh lá: đi thẳng, vùng ngoài: rẽ trái/phải.

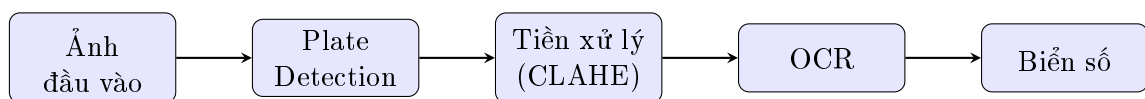
Chương 3

Cơ sở lý thuyết: Nhận diện biển số xe

3.1 Tổng quan về ALPR

Hệ thống nhận diện biển số xe tự động (Automatic License Plate Recognition - ALPR) thường bao gồm hai giai đoạn chính:

1. **Plate Detection:** Xác định vị trí biển số trong ảnh
2. **Character Recognition (OCR):** Nhận dạng các ký tự trên biển số



Hình 3.1: Pipeline tổng quan của hệ thống ALPR

3.2 Biển số xe Việt Nam

3.2.1 Các định dạng biển số

Biển số xe Việt Nam có các định dạng chính sau:

Bảng 3.1: Các định dạng biển số xe Việt Nam

Loại	Định dạng	Ví dụ
Xe ô tô (1 dòng)	XX-Y XXXXX	30G-07784
Xe ô tô (2 dòng)	XX-Y / XXXXX	30G / 07784
Xe máy	XX-YY / XXX.XX	29-H1 / 234.56

Trong đó:

- 2 chữ số đầu: Mã tỉnh/thành phố (VD: 30 = Hà Nội, 29 = Hà Nội cũ, 51 = TP.HCM)
- Chữ cái: Ký hiệu seri (A-Z, trừ I, O, Q)
- 5 chữ số cuối: Số thứ tự đăng ký

3.2.2 Thách thức nhận diện biển số Việt Nam

Việc nhận diện biển số Việt Nam gặp một số thách thức đặc thù:

- **Đa dạng định dạng:** Biển số có thể là 1 dòng hoặc 2 dòng, với các kích thước khác nhau
- **Font chữ đặc thù:** Font chữ trên biển số Việt Nam có một số điểm khác biệt
- **Điều kiện thực tế:** Biển số thường bị bẩn, mờ, hoặc nghiêng do góc quay camera
- **Ánh sáng:** Độ phản chiếu của biển số thay đổi theo điều kiện ánh sáng

3.3 PaddleOCR - Nhận dạng ký tự

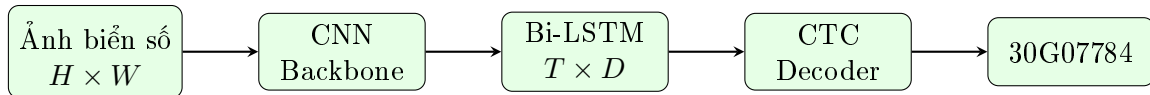
3.3.1 Giới thiệu PaddleOCR

PaddleOCR là một bộ công cụ OCR mã nguồn mở được phát triển bởi PaddlePaddle (Baidu). PaddleOCR hỗ trợ nhận dạng văn bản đa ngôn ngữ với độ chính xác cao và tốc độ xử lý nhanh, phù hợp cho các ứng dụng thời gian thực.

3.3.2 Kiến trúc CRNN

PaddleOCR sử dụng kiến trúc CRNN (Convolutional Recurrent Neural Network) bao gồm:

1. **CNN Backbone:** Trích xuất đặc trưng không gian từ ảnh đầu vào. Backbone thường sử dụng là MobileNet hoặc ResNet biến thể.
2. **Sequence Modeling (LSTM):** Mô hình hóa quan hệ tuần tự giữa các ký tự. Sử dụng Bidirectional LSTM để capture thông tin từ cả hai hướng.
3. **CTC Decoder:** Giải mã chuỗi ký tự cuối cùng từ output của LSTM.



Hình 3.2: Kiến trúc CRNN cho OCR trong PaddleOCR

3.3.3 CTC Loss

CTC (Connectionist Temporal Classification) cho phép huấn luyện mà không cần alignment giữa input và output. Đây là đặc điểm quan trọng vì ta không biết chính xác vị trí của từng ký tự trên ảnh.

Xác suất của một chuỗi output $\mathbf{l}$ được tính:

$$P(\mathbf{l}|\mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} P(\pi|\mathbf{x}) \quad (3.1)$$

Trong đó $\mathcal{B}^{-1}(\mathbf{l})$ là tập tất cả các path có thể tạo ra $\mathbf{l}$ sau khi loại bỏ ký tự blank và ký tự lặp.

Ví dụ: Với chuỗi mục tiêu “AB”, các path hợp lệ có thể là:

- “A-A-B-B” → “AB” (loại bỏ lặp)
- “A-B-” → “AB” (loại bỏ blank ‘-’)
- “-A-B-” → “AB”

CTC Loss được tính bằng negative log-likelihood:

$$\mathcal{L}_{CTC} = -\log P(\mathbf{l}|\mathbf{x}) \quad (3.2)$$

3.4 Tiền xử lý ảnh biến số

Chất lượng ảnh biến số ảnh hưởng trực tiếp đến độ chính xác của OCR. Chúng tôi áp dụng các kỹ thuật tiền xử lý sau:

3.4.1 CLAHE - Cân bằng histogram thích ứng

CLAHE (Contrast Limited Adaptive Histogram Equalization) là kỹ thuật cải thiện độ tương phản ảnh, đặc biệt hiệu quả cho ảnh có độ sáng không đều.

Nguyên lý hoạt động

1. **Chia tile:** Chia ảnh thành các tile nhỏ (VD: 8×8 pixels)
2. **Histogram Equalization cục bộ:** Thực hiện histogram equalization trên từng tile
3. **Clip Limit:** Giới hạn độ tương phản (clip limit) để tránh khuếch đại noise. Nếu histogram bin vượt quá clip limit, phần dư sẽ được phân bổ lại đều cho các bin khác.
4. **Bilinear Interpolation:** Nội suy bilinear để ghép các tile lại một cách mượt mà, tránh hiện tượng blocking artifact.

Công thức toán học

Công thức histogram equalization cho một pixel:

$$s_k = (L - 1) \cdot \sum_{j=0}^k p_r(r_j) = (L - 1) \cdot \text{CDF}(r_k) \quad (3.3)$$

Trong đó:

- s_k : Mức xám mới của pixel
- L : Số mức xám (256 cho ảnh 8-bit)
- $p_r(r_j)$: Xác suất xuất hiện của mức xám r_j
- CDF: Cumulative Distribution Function của histogram

Clip Limit trong CLAHE:

$$\beta = \frac{M \times N}{L} \times \text{clipLimit} \quad (3.4)$$

Trong đó $M \times N$ là kích thước tile và clipLimit là tham số (thường từ 2.0 đến 4.0).

3.4.2 Sharpening - Làm sắc nét

Bộ lọc làm sắc nét giúp tăng cường các cạnh của ký tự, làm cho chúng rõ ràng hơn trước khi đưa vào OCR.

Kernel Sharpening

Chúng tôi sử dụng kernel convolution để làm sắc nét:

$$K_{sharpen} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (3.5)$$

Nguyên lý

Kernel này hoạt động bằng cách:

- Nhân pixel trung tâm với hệ số lớn (5)
- Trừ đi trung bình của 4 pixel lân cận
- Kết quả: tăng cường sự khác biệt giữa pixel và các pixel xung quanh $\rightarrow$ cạnh sắc nét hơn

Công thức tính pixel mới:

$$I'(x, y) = 5 \cdot I(x, y) - [I(x-1, y) + I(x+1, y) + I(x, y-1) + I(x, y+1)] \quad (3.6)$$

3.4.3 Quy trình tiền xử lý hoàn chỉnh

1. **Crop ảnh biển số:** Từ kết quả detection của YOLOv8
2. **Chuyển Grayscale:** Để áp dụng CLAHE
3. **Áp dụng CLAHE:** Với clipLimit=2.0, tileGridSize=(8,8)
4. **Chuyển về RGB:** Để đưa vào PaddleOCR
5. **Sharpening (tùy chọn):** Áp dụng nếu ảnh còn mờ

3.5 So sánh các phiên bản YOLOv8

Trong đề tài này, chúng tôi sử dụng **YOLOv8n** (nano) cho cả phát hiện phương tiện và phát hiện biển số vì yêu cầu cân bằng giữa tốc độ và độ chính xác.

Bảng 3.2: So sánh các phiên bản YOLOv8

Model	Params (M)	FLOPs (G)	mAP@50 (COCO)
YOLOv8n	3.2	8.7	37.3
YOLOv8s	11.2	28.6	44.9
YOLOv8m	25.9	78.9	50.2
YOLOv8l	43.7	165.2	52.9
YOLOv8x	68.2	257.8	53.9

Lý do chọn YOLOv8n:

- Số lượng tham số nhỏ (3.2M) $\rightarrow$ phù hợp triển khai trên phần cứng phổ thông
- FLOPs thấp (8.7G) $\rightarrow$ tốc độ inference cao
- Đối với bài toán phát hiện biển số (single class, đối tượng tương đối dễ nhận diện), YOLOv8n đủ để đạt độ chính xác cao

3.6 Cơ chế Validation & Retry

3.6.1 Vấn đề với OCR đơn frame

OCR có thể cho kết quả khác nhau giữa các frame do:

- Biển số bị rung, blur khi xe di chuyển
- Ánh sáng thay đổi (phản chiếu, bóng)
- Góc nhìn thay đổi khi xe tiến lại gần camera
- Nhiễu từ mô hình OCR

3.6.2 Giải pháp: Format Validation

Thay vì lưu trữ nhiều kết quả và voting, chúng tôi sử dụng phương pháp validate format biển số Việt Nam:

Algorithm 1: Thuật toán Validation biển số Việt Nam

Input: *ocr_text*: Kết quả OCR, *vehicle_class*: Loại xe
Output: (*is_valid*, *cleaned_plate*): Kết quả validation

```
1 cleaned  $\leftarrow$  RemoveSpaces(ocr_text);
2 if vehicle_class  $\in$  {motorbike, bicycle} then
3   | pattern  $\leftarrow$  “^[0-9]{2}[A-Z][A-Z0-9][0-9]{4,5}$”;
4   | min_len  $\leftarrow$  8, max_len  $\leftarrow$  9;
5 else
6   | pattern  $\leftarrow$  “^[0-9]{2}[A-Z][0-9]{4,5}$”;
7   | min_len  $\leftarrow$  7, max_len  $\leftarrow$  8;
8 end
9 if min_len  $\leq$  len(cleaned)  $\leq$  max_len AND Match(pattern, cleaned)
   | then
10  | return (True, cleaned);
11 end
12 return (False, cleaned);
13
```

3.6.3 Retry Mechanism

Khi kết quả OCR không đúng format, hệ thống sẽ retry ở frame tiếp theo:

- Chỉ lưu kết quả **đã validate** (đúng format)
- Nếu OCR sai format hoặc không đọc được $\rightarrow$ không lưu, thử lại frame sau
- Khi đã có kết quả hợp lệ $\rightarrow$ không xử lý OCR nữa (tiết kiệm tài nguyên)

$$\text{ShouldRunOCR}(\text{track_id}) = \begin{cases} \text{True} & \text{nếu chưa có kết quả valid} \\ \text{False} & \text{nếu đã có kết quả valid} \end{cases} \quad (3.7)$$

3.7 Character Correction cho biển số Việt Nam

3.7.1 Các lỗi phổ biến của OCR

Do một số ký tự có hình dáng tương tự, PaddleOCR có thể nhầm lẫn:

Bảng 3.3: Bảng hiệu chỉnh ký tự phổ biến

Sai	Đúng	Điều kiện
0 (số không)	O (chữ O)	Vị trí chữ cái (index 2)
O (chữ O)	0 (số không)	Vị trí chữ số
1 (số một)	I (chữ I)	Vị trí chữ cái
8 (số tám)	B (chữ B)	Vị trí chữ cái
5 (số năm)	S (chữ S)	Vị trí chữ cái

3.7.2 Logic hiệu chỉnh dựa trên định dạng

Dựa trên cấu trúc biển số Việt Nam (ví dụ: 30G-07784):

- **Vị trí 0-1:** Bắt buộc là chữ số (mã tỉnh)
- **Vị trí 2:** Bắt buộc là chữ cái (seri)
- **Vị trí 3-7:** Bắt buộc là chữ số (số thứ tự)

Thuật toán hiệu chỉnh:

1. Kiểm tra độ dài chuỗi OCR
2. Tại mỗi vị trí, kiểm tra xem ký tự có đúng loại (chữ/số) không
3. Nếu sai loại, tra bảng mapping để sửa

3.8 Hai phương pháp triển khai ALPR

Trong hệ thống của chúng tôi, có hai cách tiếp cận để thực hiện nhận diện biển số xe:

3.8.1 Phương pháp 1: Two-Stage ALPR (Hai giai đoạn)

Đây là phương pháp truyền thống, chia quá trình thành hai bước độc lập:

Kiến trúc

1. **Giai đoạn 1 - Phát hiện phương tiện:** Sử dụng YOLOv8n (5 lớp) để phát hiện các loại xe (xe máy, xe đạp, ô tô, xe buýt, xe tải)
2. **Giai đoạn 2 - Phát hiện biển số:** Crop vùng xe từ giai đoạn 1, chạy qua YOLOv8n chuyên biệt (1 lớp: license_plate) để phát hiện biển số
3. **Giai đoạn 3 - OCR:** Crop vùng biển số, áp dụng tiền xử lý (CLAHE, sharpening), sau đó sử dụng PaddleOCR để nhận dạng ký tự

Ưu điểm

- **Độ chính xác cao:** Mô hình chuyên biệt cho biển số đạt $mAP@50 = 99.5\%$
- **Phát hiện biển nhỏ tốt:** Vì chỉ focus vào 1 task (plate detection), mô hình học rất tốt các đặc trưng của biển số

- **Linh hoạt:** Có thể thay thế từng module độc lập (ví dụ: thay YOLOv8 bằng Faster R-CNN cho plate detection)

Nhược điểm

- **Chậm:** Cần 2 lần inference qua YOLO (vehicle detection + plate detection), tốc độ chỉ đạt 5–10 FPS
- **Tiêu tốn tài nguyên:** Phải load 2 models YOLOv8 vào GPU/RAM (2.5GB)
- **Phức tạp:** Logic phải quản lý 2 models, crop vùng xe, rồi mới detect plate

3.8.2 Phương pháp 2: One-Stage Semi-supervised (Một giai đoạn)

Phương pháp này tích hợp việc phát hiện phương tiện và biển số vào một mô hình duy nhất.

Kiến trúc

1. **Giai đoạn 1 - Phát hiện đa lớp:** Sử dụng YOLOv8n (6 lớp) để phát hiện đồng thời cả xe và biển số trong một lần forward pass
2. **Giai đoạn 2 - Matching:** Sử dụng thuật toán containment để gán biển số cho xe (kiểm tra xem bbox biển có nằm trong bbox xe không)
3. **Giai đoạn 3 - OCR:** Tương tự Phương pháp 1, crop vùng biển số và nhận dạng ký tự

Quy trình gán nhãn bán giám sát (Semi-supervised)

Để có được mô hình 6 lớp, chúng tôi áp dụng kỹ thuật Semi-supervised Labeling:

1. **Chuẩn bị Teacher Model:** Sử dụng mô hình phát hiện biển số từ Phương pháp 1 (mAP@50 = 99.5%) làm teacher
2. **Tự động gán nhãn:** Chạy teacher model trên toàn bộ dataset vietnam_vehicle_dataset (chỉ có 5 lớp xe) để tự động phát hiện và gán nhãn biển số
3. **Hợp nhất dataset:** Merge các annotation biển số vào file gốc, tạo thành dataset 6 lớp (5 loại xe + license_plate)
4. **Huấn luyện lại:** Train YOLOv8n từ đầu trên dataset 6 lớp này

Ưu điểm

- **Nhanh:** Chỉ cần 1 lần inference qua YOLO, tốc độ đạt 15–20 FPS
- **Tiết kiệm tài nguyên:** Chỉ cần load 1 model (1.2GB)
- **Mapping tự động:** Dễ dàng gán biển số cho xe thông qua kiểm tra containment
- **Học ngữ cảnh:** Mô hình học được mối quan hệ không gian giữa xe và biển số

Nhược điểm

- **Độ chính xác thấp hơn:** mAP@50 cho tất cả lớp chỉ đạt 78.5% (do class imbalance)
- **Khó phát hiện biển nhỏ:** Biển số chiếm tỷ lệ nhỏ trong dataset, dễ bị miss khi xe ở xa

- **Phụ thuộc teacher model:** Chất lượng annotation tự động phụ thuộc vào độ chính xác của teacher model

3.8.3 So sánh hai phương pháp

Bảng 3.4: So sánh hai phương pháp triển khai ALPR

Tiêu chí	Phương pháp 1 (Two-Stage)	Phương pháp 2 (One-Stage)
Số model	2 (Vehicle + Plate)	1 (6-class)
Tốc độ	Chậm (5–10 FPS)	Nhanh (15–20 FPS)
Độ chính xác plate	Cao (99.5%)	Trung bình (78.5%)
Tài nguyên	Nhiều (2.5GB)	Ít (1.2GB)
Triển khai	Phức tạp	Đơn giản
Sử dụng tốt nhất	Yêu cầu độ chính xác cao, camera tốt	Ưu tiên tốc độ, real-time

Trong thực tế, chúng tôi triển khai cả hai phương pháp và cho phép người dùng lựa chọn phù hợp với nhu cầu của hệ thống (độ chính xác cao vs. tốc độ nhanh).

Chương 4

Phương pháp thực hiện

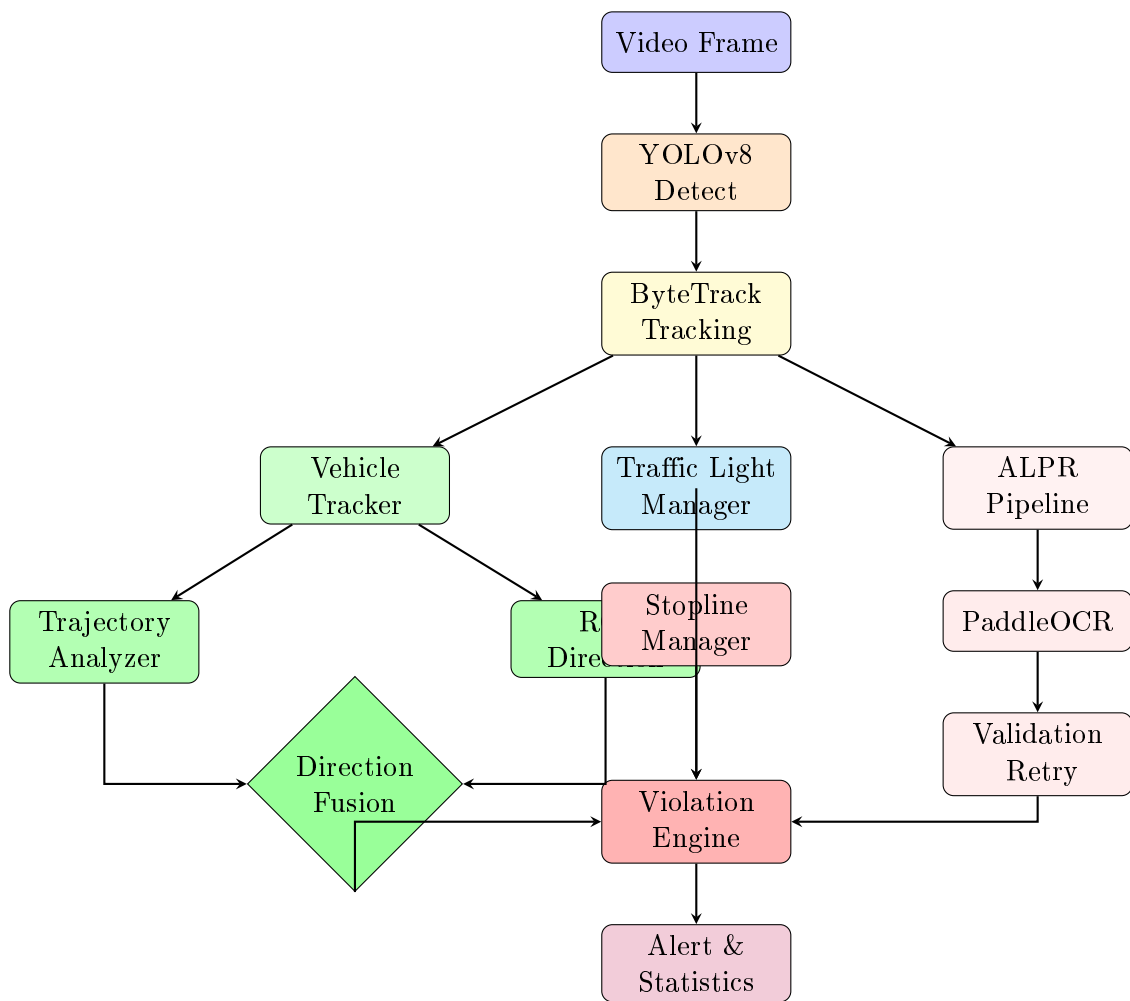
Chương này trình bày chi tiết phương pháp triển khai toàn bộ hệ thống, từ kiến trúc tổng thể, chuẩn bị dữ liệu, huấn luyện mô hình cho đến các thuật toán xác định hướng di chuyển, phát hiện vi phạm và nhận diện biển số xe.

4.1 Kiến trúc tổng thể của hệ thống

Hệ thống được thiết kế theo mô hình xử lý đa luồng sử dụng PyQt5 QThread để đảm bảo khả năng xử lý thời gian thực. Quy trình xử lý chính được chia thành các module chuyên biệt tương tác với nhau thông qua Global State quản lý bởi VideoThread.

Giải thích các module chính:

- **YOLOv8 Detect:** Phát hiện phương tiện và biển số (tùy phương pháp 1 hoặc 2)
- **ByteTrack Tracking:** Gán ID duy nhất và theo dõi xe qua các frame
- **Vehicle Tracker:** Lưu lịch sử vị trí xe (position history) để phân tích hướng
 - *Trajectory Analyzer:* Tính hướng dựa trên lịch sử di chuyển và reference vector
 - *ROI Direction:* Tính hướng dựa trên vị trí xe trong ROI
 - *Direction Fusion:* Kết hợp 2 phương pháp để ra hướng cuối cùng (8 hướng)
- **Traffic Light Manager:** Phát hiện màu đèn giao thông (HSV color filtering trong ROI)
- **ALPR Pipeline:** Nhận diện biển số
 - *PaddleOCR:* Đọc ký tự từ ảnh biển số (sau tiền xử lý CLAHE)
 - *Validation & Retry:* Kiểm tra định dạng biển số Việt Nam, retry nếu sai
- **Stopline Manager:** Kiểm tra xe có vượt qua vạch dừng hay không
- **Violation Engine:** Logic tổng hợp kiểm tra vi phạm (đèn đỏ + stopline, sai làn, sai hướng)



Hình 4.1: Sơ đồ luồng xử lý chi tiết với cơ chế Direction Fusion và ALPR

4.2 Chuẩn bị dữ liệu phát hiện phương tiện

4.2.1 Quy trình thu thập dữ liệu

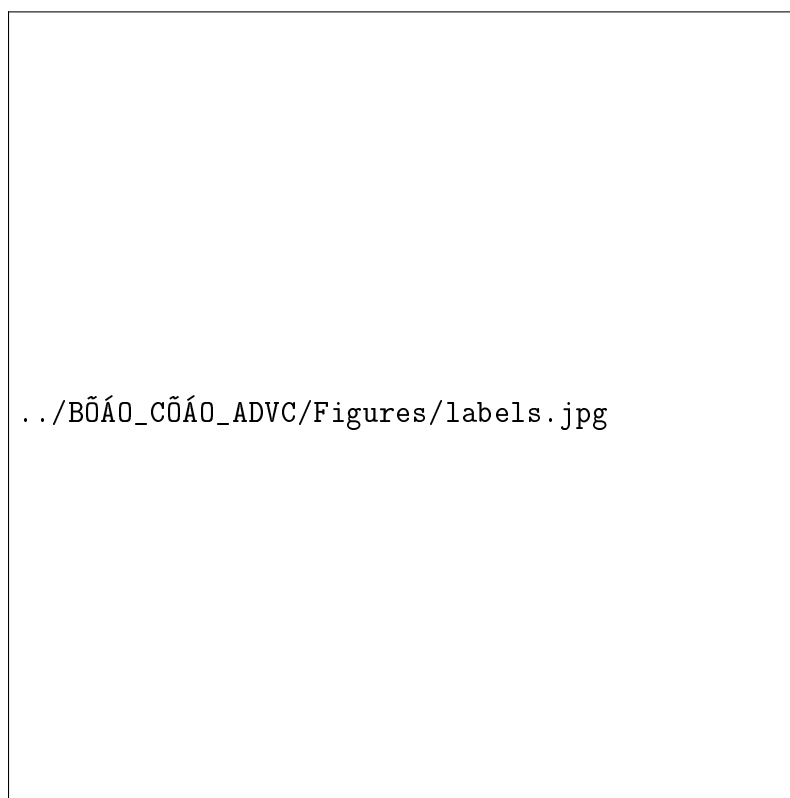
Thu thập dữ liệu thô

Dữ liệu video và hình ảnh được thu thập từ hai nguồn chính:

- **Tự quay chụp thực tế:** Nhóm đã sử dụng camera điện thoại để quay video tại các nút giao thông trọng điểm, các ngã tư có đèn tín hiệu và đường vành đai vào các khung giờ, đảm bảo sự đa dạng về điều kiện ánh sáng (sáng sớm, trưa nắng, chiều tà) và góc nhìn.
- **Sưu tầm bổ sung:** Để làm phong phú thêm các trường hợp hiếm, chúng tôi bổ sung thêm một phần dữ liệu từ các video camera giao thông công cộng (CCTV).

Gán nhãn dữ liệu

Sau khi trích xuất video thành các khung hình ảnh, nhóm thực hiện gán nhãn thủ công sử dụng công cụ **CVAT** và **Roboflow**.



Hình 4.2: Phân bố số lượng nhãn và vị trí không gian của các đối tượng

- **Quy mô:** Tổng cộng đã gán nhãn cho hơn **16.000 ảnh** với hơn **65.000 bounding boxes**.
- **Định dạng:** Nhãn được xuất ra dưới định dạng YOLO chuẩn, bao gồm 5 lớp đối tượng:
 1. **xe may:** Bao gồm xe máy số, xe tay ga và xe máy điện (chiếm tỷ trọng

lớn nhất).

2. **o to**: Ô tô con, xe taxi, xe bán tải.
3. **xe bus**: Xe buýt nội đô và xe khách.
4. **xe tai**: Xe tải nhẹ và xe tải nặng.
5. **xe đạp**: Xe đạp và xe đạp điện.

Quy trình gán nhãn tuân thủ nguyên tắc: chỉ gán nhãn các đối tượng nhìn thấy rõ ít nhất 30%, các đối tượng quá mờ hoặc bị che khuất quá nhiều sẽ được bỏ qua để tránh làm nhiễu mô hình.

Từ biểu đồ phân bố (Hình 4.2), ta nhận thấy sự mất cân bằng dữ liệu đặc trưng: lớp 'Xe máy' và 'Ô tô' chiếm tỷ trọng lớn, trong khi 'Xe buýt' và 'Xe đạp' chiếm thiểu số. Kích thước bounding box tập trung nhiều ở vùng nhỏ và trung tâm, phản ánh các đối tượng xe cộ thường xuất hiện ở xa và đi vào giữa khung hình từ góc quay camera giám sát.

4.2.2 Tiền xử lý dữ liệu

Trước khi đưa vào huấn luyện, dữ liệu hình ảnh được xử lý qua các bước:

- **Auto-Orientation**: Điều chỉnh hướng ảnh dựa trên thông tin EXIF.
- **Resize**: Đưa ảnh về kích thước tiêu chuẩn 416×416 pixels. Đây là kích thước tối ưu được chúng tôi lựa chọn để cân bằng giữa tốc độ xử lý và độ chính xác.
- **Augmentation**: Sử dụng kỹ thuật Mosaic Augmentation (ghép 4 ảnh huấn luyện thành 1) trong quá trình train của YOLOv8. Kỹ thuật này giúp mô hình học được các đối tượng ở nhiều tỷ lệ kích thước khác nhau và tăng khả năng nhận diện trong các ngữ cảnh phức tạp.

4.3 Huấn luyện mô hình phát hiện phương tiện

4.3.1 Cấu hình huấn luyện

Chúng tôi sử dụng mô hình cơ sở (pre-trained) `yolo8n.pt` (phiên bản nano) được huấn luyện trước trên bộ dữ liệu COCO. Việc sử dụng Transfer Learning giúp mô hình hội tụ nhanh hơn và đạt độ chính xác tốt hơn với lượng dữ liệu giới hạn.

Các tham số huấn luyện chính được thiết lập như sau:

Bảng 4.1: Tham số cấu hình huấn luyện YOLOv8 cho phát hiện phương tiện

Tham số	Giá trị
Kích thước ảnh	416
Số lượng Epochs	100
Batch Size	16
Optimizer	SGD
Initial Learning Rate	0.01
Momentum	0.937
Weight Decay	0.0005
Số lớp	5 (xe máy, ô tô, xe buýt, xe tải, xe đạp)

4.3.2 Quá trình tối ưu hóa

Trong quá trình huấn luyện, hàm mất mát tổng hợp bao gồm Box Loss, Class Loss và DFL Loss được theo dõi sát sao. Chúng tôi áp dụng kỹ thuật **Early Stopping** để dừng huấn luyện nếu chỉ số mAP@50 trên tập validation không cải thiện sau 50 epochs liên tiếp, nhằm tránh hiện tượng quá khớp (overfitting).

4.4 Thuật toán xác định hướng di chuyển

Khác với các phương pháp truyền thống chỉ dựa vào sự thay đổi tọa độ đơn thuần, hệ thống đề xuất sử dụng cơ chế Direction Fusion kết hợp hai nguồn thông tin để đạt độ chính xác cao nhất ngay cả khi camera bị nghiêng hoặc xe di chuyển phức tạp.

4.4.1 Phân tích quỹ đạo với Reference Vector (Trajectory-based)

Module `TrajectoryDirectionAnalyzer` xác định hướng di chuyển bằng cách so sánh vector chuyển động của xe với vector tham chiếu của đường. Phương pháp này giúp loại bỏ sai số do góc quay của camera.

1. **Thiết lập Reference Vector ($\vec{V}_{ref}$):** Vector này được xác định thủ công hoặc tự động, đại diện cho hướng "đi thẳng" trên khung hình (ví dụ: vector dọc theo làn đường).
2. **Tính toán Motion Vector ($\vec{V}_{motion}$):** Được tính từ N vị trí lịch sử gần nhất của xe (sliding window size = 15).
3. **Tính góc lệch tương đối (θ):** Sử dụng tích vô hướng (Dot Product) và tích có hướng (Cross Product) để xác định góc lệch:

$$\theta = \text{atan2}(\vec{V}_{motion} \times \vec{V}_{ref}, \vec{V}_{motion} \cdot \vec{V}_{ref}) \quad (4.1)$$

Kết quả θ (đổi sang độ) được dùng để phân loại với ngưỡng 50° :

- $|\theta| \leq 50^\circ$: Đi thẳng (STRAIGHT).
- $\theta > 50^\circ$: Rẽ phải (RIGHT).
- $\theta < -50^\circ$: Rẽ trái (LEFT).

4.4.2 Xác định hướng dựa trên vùng quan tâm (ROI-based)

Module `ROIDirectionManager` sử dụng các đa giác (polygons) được định nghĩa trước trên mặt đường. Khi trọng tâm của xe nằm trong vùng ROI nào, xe sẽ được gán nhãn hướng tương ứng của vùng đó (ví dụ: Làn rẽ trái, Làn đi thẳng). Phương pháp này hoạt động tốt ngay cả khi xe di chuyển chậm hoặc đứng yên chờ đèn.

4.4.3 Cơ chế kết hợp (Direction Fusion)

Module `DirectionFusion` tổng hợp kết quả từ hai nguồn trên để đưa ra quyết định cuối cùng, giải quyết bài toán xung đột dữ liệu:

Algorithm 2: Thuật toán Direction Fusion

Input: D_{traj} (Trajectory Direction), C_{traj} (Confidence), D_{roi} (ROI Direction)
Output: D_{final} (Final Direction)

```
1 if  $D_{roi}$  is None and  $D_{traj}$  is Unknown then
2 |   return Unknown;
3 else if  $D_{roi}$  exists and  $D_{traj}$  is Unknown then
4 |   return  $D_{roi}$ ;
5 |   ; // Fallback to ROI
6 else if Conflict exists ( $D_{roi} \neq D_{traj}$ ) then
7 |   if  $C_{traj} \geq 0.5$  then
8 |   |   return  $D_{traj}$ ;
9 |   |   ; // Trajectory priority if confident
10 |  else
11 |  |  return  $D_{roi}$ ;
12 |  |  ; // ROI priority if trajectory noisy
13 |  end
14 else
15 |  return  $D_{roi}$  (matches  $D_{traj}$ );
16 end
```

4.5 Hệ thống phát hiện vi phạm

Hệ thống sử dụng máy trạng thái (state machine) để kiểm tra vi phạm, đảm bảo mỗi phương tiện chỉ bị xử lý một lần tại thời điểm cắt qua vạch dừng.

4.5.1 Quản lý tín hiệu đèn giao thông

Module `TrafficLightManager` sử dụng không gian màu HSV để phát hiện trạng thái đèn. Các ngưỡng màu được tinh chỉnh để phân biệt rõ ba trạng thái:

- **Đỏ:** Hue $\in [0, 10] \cup [160, 180]$.
- **Vàng:** Hue $\in [15, 35]$.
- **Xanh:** Hue $\in [40, 90]$.

Hệ thống phân biệt được 4 loại đèn: Đèn tròn, Đèn rẽ trái, Đèn rẽ phải và Đèn đi thẳng.

4.5.2 Logic phát hiện vượt đèn đỏ

Logic kiểm tra vi phạm (`ViolationChecker`) được kích hoạt khi sự kiện *Stopline Crossing* xảy ra. Thuật toán kiểm tra tính hợp lệ dựa trên sự kết hợp giữa hướng di chuyển của xe và trạng thái của tất cả các đèn hiện có.

4.5.3 Logic phát hiện sai làn đường

Module `LaneManager` kiểm tra vị trí tâm xe so với các đa giác làn đường đã định nghĩa.

- Sử dụng thuật toán `pointInPolygon` (Ray Casting).

Algorithm 3: Logic kiểm tra vượt đèn đỏ

Input: Vehicle Direction D_{veh} , Traffic Lights TLs
Output: IsViolation, Reason

```
1  $has\_green\_arrow \leftarrow \text{False}$ ;  
2  $has\_red\_arrow \leftarrow \text{False}$ ;  
3  $any\_red \leftarrow \text{False}$ ;  
4 foreach  $light$  in  $TLs$  do  
5   if  $light.color$  is GREEN then  
6     if  $light$  matches  $D_{veh}$  (Arrow or Circle) then  
7        $has\_green\_arrow \leftarrow \text{True}$ ;  
8     end  
9   end  
10  if  $light.color$  is RED then  
11     $any\_red \leftarrow \text{True}$ ;  
12    if  $light$  explicitly forbids  $D_{veh}$  then  
13       $has\_red\_arrow \leftarrow \text{True}$ ;  
14    end  
15  end  
16 end  
17 if  $has\_green\_arrow$  then  
18   return False (Allowed);  
19 else if  $has\_red\_arrow$  then  
20   return True (Explicit Prohibition);  
21 else if  $any\_red$  and not  $has\_green\_arrow$  then  
22   return True (Red Light Violation);  
23 else  
24   return False (No clear violation);  
25 end
```

- Mỗi làn đường có danh sách `allowed_labels` (ví dụ: ['car', 'bus']).
- Nếu loại phương tiện của xe không nằm trong danh sách cho phép của làn đường hiện tại $\rightarrow$ Vi phạm sai làn.

4.5.4 Điểm chạm tham chiếu (Reference Point)

Khi tính toán vi phạm (dù là đè vạch dừng hay lấn làn), tuyệt đối không dùng tâm của Bbox. Cần sử dụng **trung điểm cạnh đáy** của Bbox – tương ứng với vị trí bánh xe chạm đất. Điều này giúp loại bỏ lỗi dương tính giả (false positive) khi xe nghiêng hoặc do góc quay camera.

4.5.5 Ngưỡng dung sai

Nên thiết lập vùng đệm an toàn. Ví dụ: Xe chỉ bị coi là vi phạm nếu điểm chạm lấn sang vùng cấm quá 10–15%, hoặc lấn quá một số pixel nhất định. Điều này giúp hệ thống "mềm dẻo" hơn trước các sai số do rung lắc camera hoặc bounding box bị lệch nhẹ.

4.6 Pipeline nhận diện biển số xe (ALPR)

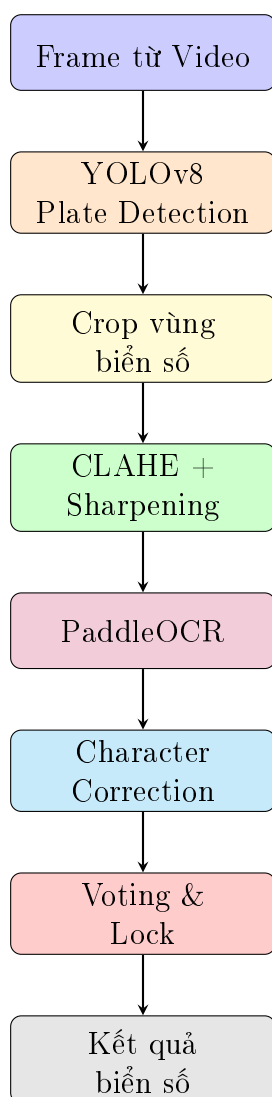
Chúng tôi đề xuất hai phương pháp tiếp cận để giải quyết bài toán phát hiện biển số:

1. **Phương pháp 1 (Hai giai đoạn - Two Stage):** Sử dụng mô hình chuyên biệt để phát hiện biển số sau khi đã phát hiện phương tiện.
2. **Phương pháp 2 (Một giai đoạn - One Stage):** Tích hợp nhận biển số vào mô hình phát hiện phương tiện để phát hiện đồng thời cả xe và biển số.

4.6.1 Phương pháp 1: Mô hình phát hiện biển số chuyên biệt

Tổng quan Pipeline

Hệ thống nhận diện biển số được xây dựng theo kiến trúc Two-Stage với các bước:



Hình 4.3: Pipeline xử lý nhận diện biển số xe

Plate Detection với YOLOv8

Chúng tôi huấn luyện mô hình YOLOv8n trên bộ dữ liệu VNLP (Vietnamese License Plate) với cấu hình:

Bảng 4.2: Cấu hình huấn luyện Plate Detection

Tham số	Giá trị
Base model	YOLOv8n (pretrained)
Input size	416×416
Batch size	16
Epochs	100
Optimizer	AdamW
Learning rate	0.002 (cosine decay)
Số lớp	1 (license_plate)

Bộ dữ liệu VNLP

Bộ dữ liệu VNLP bao gồm các ảnh biển số xe Việt Nam được gắn nhãn với định dạng YOLO:

- Tổng số ảnh huấn luyện: 5000 ảnh
- Đa dạng điều kiện: Ban ngày, ban đêm, góc nghiêng
- Các loại biển: Xe ô tô, xe máy, biển trắng, biển vàng

4.6.2 Phương pháp 2: Tích hợp phát hiện đa lớp (Semi-supervised)

Để tối ưu hóa thời gian xử lý và tận dụng ngữ cảnh xe, chúng tôi đề xuất phương pháp thứ hai: huấn luyện một mô hình duy nhất có khả năng phát hiện đồng thời phương tiện và biển số.

Tổng quan Pipeline

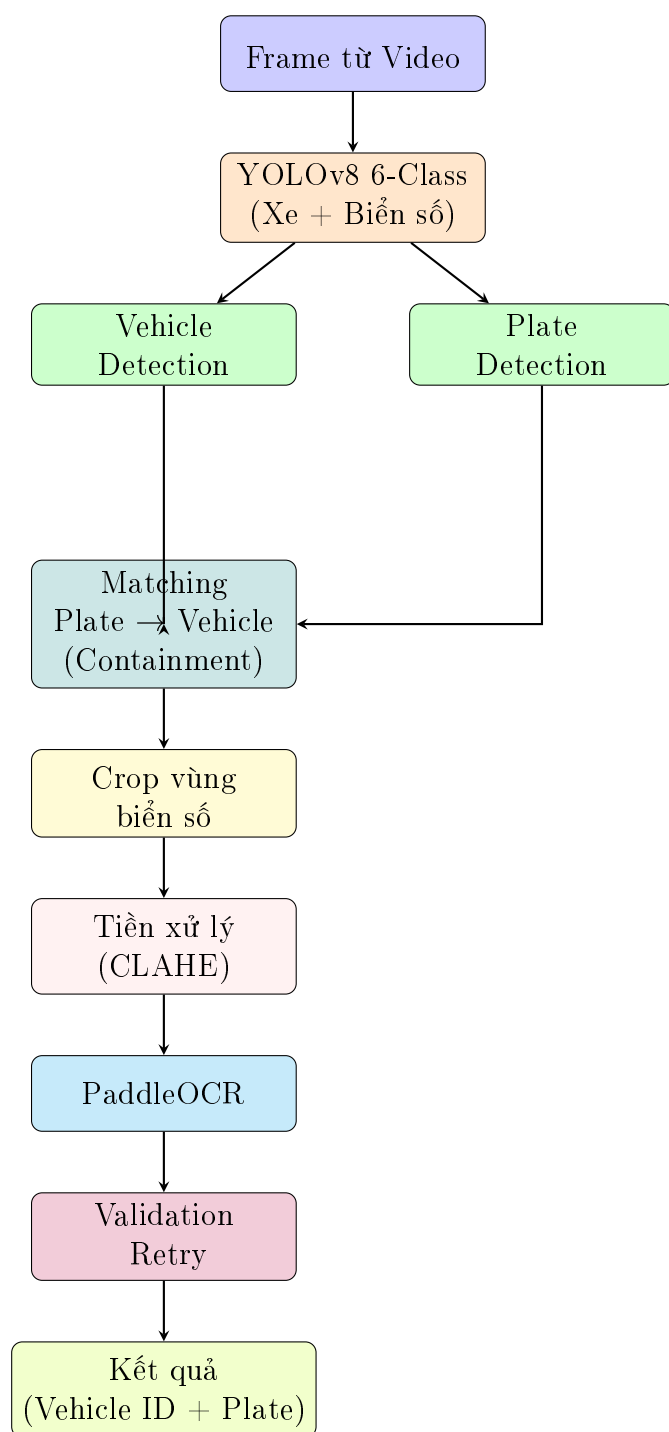
Ưu điểm của phương pháp này:

- **Hiệu quả:** Chỉ cần 1 lần forward pass qua YOLOv8 để có cả xe và biển số
- **Ngữ cảnh:** Mô hình học được mối quan hệ vị trí giữa xe và biển số
- **Tốc độ:** Giảm độ trễ inference so với phương pháp 2 giai đoạn
- **Mapping tự động:** Dễ dàng liên kết biển số với xe thông qua kiểm tra containment (biển số nằm 100% trong bbox xe)

Quy trình gắn nhãn bán giám sát (Semi-supervised Labeling)

Do bộ dữ liệu phương tiện gốc (`vietnam_vehicle_dataset`) chỉ chứa 5 nhãn phương tiện, chúng tôi thực hiện quy trình gắn nhãn tự động để bổ sung nhãn biển số:

1. **Bước 1 - Chuẩn bị mô hình Teacher:** Sử dụng mô hình phát hiện biển số (Model 1 - đã huấn luyện trên VNLP) làm nòng cốt.
2. **Bước 2 - Inference gắn nhãn:** Chạy Model 1 trên toàn bộ tập ảnh của `vietnam_vehicle_dataset`.



Hình 4.4: Pipeline Phương pháp 2: Phát hiện đồng thời xe và biển số trong một mô hình

3. **Bước 3 - Hợp nhất nhãn:** Bổ sung các bounding box biển số (license_plate) vào các file annotation gốc. Nâng tổng số lớp từ 5 lên 6 (5 loại xe + license_plate).

Huấn luyện mô hình tích hợp

Sau khi có bộ dữ liệu 6 lớp, chúng tôi tiến hành huấn luyện lại từ đầu với cấu hình tương tự:

- **Model:** YOLOv8n
- **Epochs:** 100
- **Image Size:** 416 pixel
- **Classes:** 6 (Xe máy, Xe đạp, Ô tô, Xe buýt, Xe tải, Biển số)

Phương pháp này giúp mô hình học được mối tương quan vị trí giữa xe và biển số, đồng thời giảm độ trễ inference (do chỉ cần 1 lần forward pass).

4.7 Tiền xử lý ảnh biển số

Sau khi crop vùng biển số, chúng tôi áp dụng các bước tiền xử lý:

4.7.1 Bước 1: CLAHE (Contrast Limited Adaptive Histogram Equalization)

Áp dụng CLAHE để cải thiện độ tương phản của ảnh biển số:

- Chuyển ảnh sang grayscale
- Áp dụng CLAHE với clipLimit=2.0 và tileGridSize=(8,8)
- Chuyển lại về ảnh màu để đưa vào OCR

4.7.2 Bước 2: Sharpening

Áp dụng bộ lọc làm sắc nét để tăng cường cạnh của các ký tự, sử dụng kernel sharpening chuẩn:

$$K_{sharpen} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (4.2)$$

4.8 OCR với PaddleOCR

4.8.1 Cấu hình PaddleOCR

PaddleOCR được khởi tạo với các tham số:

- use_angle_cls=False: Biển số đã được crop đúng hướng
- lang='en': Ký tự biển số là alphanumeric
- use_gpu=True: Tăng tốc với GPU

4.8.2 Character Correction

Do PaddleOCR có thể nhầm lẫn một số ký tự tương tự, chúng tôi áp dụng bước hiệu chỉnh:

Bảng 4.3: Bảng hiệu chỉnh ký tự

Sai	Đúng	Giải thích
0 (số)	O (chữ)	Vị trí chữ cái (index 2)
O (chữ)	0 (số)	Vị trí chữ số
1 (số)	I (chữ)	Hiếm gặp
8 (số)	B (chữ)	Vị trí chữ cái

Logic hiệu chỉnh dựa trên định dạng biển số Việt Nam:

- Vị trí 0-1: Chữ số (mã tỉnh)
- Vị trí 2: Chữ cái (seri)
- Vị trí 3-7: Chữ số (số thứ tự)

4.9 Validation & Retry Mechanism

4.9.1 Vấn đề

OCR có thể cho kết quả khác nhau giữa các frame do:

- Biển số bị rung, blur
- Ánh sáng thay đổi
- Góc nhìn thay đổi khi xe di chuyển
- Nhiều từ mô hình OCR

4.9.2 Giải pháp: Format Validation

Thay vì sử dụng voting mechanism phức tạp, chúng tôi áp dụng validation dựa trên format chuẩn biển số Việt Nam:

- **Xe máy** (cls_id 2, 3): Format xxYZxxxxx (8-9 ký tự)
 - xx: 2 chữ số đầu (mã tỉnh)
 - Y: 1 chữ cái (seri)
 - Z: 1 chữ số hoặc chữ cái
 - xxxxx: 4-5 chữ số cuối
 - Ví dụ: 29B12345, 30M04019
- **Ô tô** (cls_id 0, 1, 4): Format xxYxxxxx (7-8 ký tự)
 - xx: 2 chữ số đầu (mã tỉnh)
 - Y: 1 chữ cái (seri)
 - xxxxx: 4-5 chữ số cuối
 - Ví dụ: 51A1234, 30H12345

4.9.3 Retry Mechanism

Khi OCR không cho kết quả hợp lệ, hệ thống tự động retry:

- Chỉ lưu kết quả **đúng format** (validated)

- Nếu OCR trả về text không khớp pattern → không lưu, thử lại frame tiếp theo
- Nếu đã có kết quả hợp lệ → dừng OCR cho xe đó (tiết kiệm tài nguyên)
- Log chi tiết: `VALID`, `INVALID FORMAT`, `NO TEXT`

Điều này giúp:

- Loại bỏ kết quả nhiễu ngay từ đầu
- Đảm bảo chỉ hiển thị biển số đúng format
- Giảm độ phức tạp so với voting mechanism
- Tận dụng khả năng tracking để retry tự động

4.10 Tích hợp với Vehicle Tracking

Pipeline biển số được tích hợp với hệ thống tracking để:

- Chỉ xử lý OCR cho xe có `track_id` mới hoặc chưa có kết quả valid
- Liên kết biển số với từng xe cụ thể thông qua `track_id`
- Tự động dừng OCR khi đã có kết quả đúng format để tiết kiệm tài nguyên
- Sử dụng format validation để đảm bảo chất lượng kết quả

4.11 Tổng kết phương pháp

Hệ thống hoàn chỉnh được xây dựng với các thành phần chính:

1. **Detection:** YOLOv8n cho cả phương tiện (5 lớp) và biển số (1 lớp)
2. **Tracking:** ByteTrack với Kalman Filter để theo dõi liên tục
3. **Direction Analysis:** Direction Fusion kết hợp Trajectory + ROI
4. **Violation Detection:** State machine với logic đèn + làn đường
5. **ALPR:** PaddleOCR với Character Correction + Voting & Lock

Toàn bộ pipeline được tích hợp trong ứng dụng PyQt5 với giao diện đồ họa thân thiện, cho phép người dùng cấu hình ROI, vạch dừng, làn đường một cách trực quan.

Chương 5

Thực nghiệm và Đánh giá kết quả

Chương này trình bày chi tiết môi trường thực nghiệm, thiết lập huấn luyện, kết quả đạt được và phân tích đánh giá cho cả hai thành phần chính: (1) Phát hiện và theo dõi phương tiện, (2) Nhận diện biển số xe.

5.1 Môi trường thực nghiệm

5.1.1 Cấu hình phần cứng

Các thực nghiệm trong nghiên cứu này được triển khai trên hai nền tảng: đám mây Kaggle Notebooks cho huấn luyện và máy tính cục bộ cho inference.

Bảng 5.1: Cấu hình chi tiết môi trường thực nghiệm

Thành phần	Thông số kỹ thuật
<i>Huấn luyện (Kaggle Cloud)</i>	
CPU	4 Logical Cores
RAM	31.35 GB
GPU	NVIDIA Tesla P100-PCIE (16GB VRAM)
Dung lượng đĩa	19.52 GB
<i>Inference (Local Machine)</i>	
CPU	Intel Core i5-12400
RAM	16GB DDR4
GPU	NVIDIA RTX 3060 (12GB VRAM)

5.1.2 Cấu hình phần mềm

- Ngôn ngữ lập trình: Python 3.10/3.11
- Nền tảng tính toán song song: CUDA 11.8/12.4
- Framework Deep Learning: PyTorch 2.0
- Ultralytics YOLOv8: Version 8.0.196
- PaddleOCR: Version 2.7.0
- OpenCV: Version 4.8.0
- PyQt5: Giao diện người dùng

5.2 Kết quả huấn luyện mô hình phát hiện phương tiện

5.2.1 Thiết lập huấn luyện

Chiến lược tăng cường dữ liệu

Hệ thống áp dụng kỹ thuật **Mosaic Augmentation** (ghép 4 ảnh ngẫu nhiên thành 1) trong quá trình huấn luyện (xem Hình 5.1). Kỹ thuật này đặc biệt hiệu quả cho bài toán giao thông vì:

- Giúp mô hình học được các ngữ cảnh phức tạp và các vật thể bị che khuất một phần.
- Cải thiện khả năng phát hiện các đối tượng nhỏ (như xe máy, xe đạp ở xa) bằng cách thay đổi tỷ lệ ảnh.



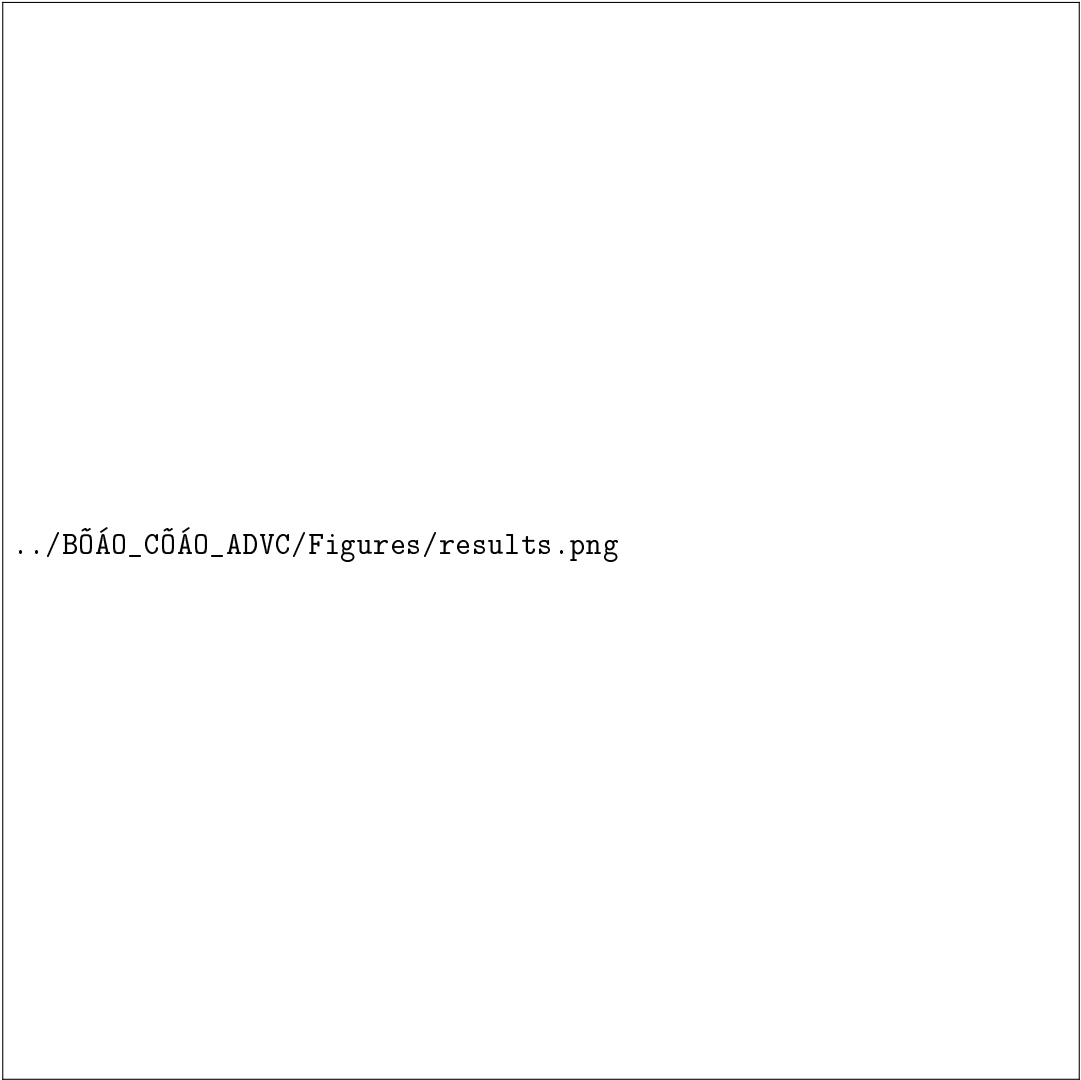
Hình 5.1: Dữ liệu huấn luyện sau khi áp dụng Mosaic Augmentation

5.2.2 Phân tích quá trình hội tụ

Biểu đồ 5.2 thể hiện diễn biến của các hàm mất mát và chỉ số mAP qua 100 epochs.

Nhận xét:

- **Sự hội tụ:** Các hàm mất mát (`box_loss`, `cls_loss`) giảm nhanh trong 20 epoch đầu và tiếp tục giảm nhẹ, ổn định dần về phía cuối chu kỳ huấn luyện. Không xuất hiện hiện tượng *overfitting* khi loss của tập validation bám sát tập train.
- **Hiệu năng:** Chỉ số mAP@50 tăng trưởng mạnh và đạt trạng thái bão hòa từ epoch thứ 80 trở đi, cho thấy việc dừng huấn luyện tại 100 epoch là hợp lý.



`../BỔ_ÁO_CỔ_ÁO_ADV_C/Figures/results.png`

Hình 5.2: Đồ thị Loss và mAP trong quá trình huấn luyện phát hiện phương tiện

5.2.3 Kết quả định lượng

Tại epoch tốt nhất (Epoch 99), mô hình đạt được các chỉ số hiệu năng ấn tượng trên tập Validation:

Bảng 5.2: Kết quả đánh giá hiệu năng mô hình YOLOv8n phát hiện phương tiện

Chỉ số	Precision(P)	Recall(R)	mAP@50	mAP@50:95
Giá trị	0.872	0.781	0.861	0.704

Kết quả chi tiết theo từng lớp

Phân tích chi tiết theo từng lớp đối tượng:

Bảng 5.3: Kết quả mAP@50 theo từng lớp phương tiện

Lớp	mAP@50	Precision	Recall
Ô tô (o to)	0.922	0.91	0.85
Xe buýt (xe bus)	0.897	0.88	0.80
Xe máy (xe may)	0.872	0.85	0.79
Xe tải (xe tai)	0.832	0.84	0.75
Xe đạp (xe dap)	0.782	0.78	0.72

- Lớp "**Ô tô**"(o to) đạt hiệu quả cao nhất với **mAP@50 = 0.922**.
- Lớp "**Xe buýt**"(xe bus) đạt **0.897** và "**Xe máy**"(xe may) đạt **0.872**.
- Lớp "**Xe đạp**"(xe dap) có hiệu năng thấp nhất (**0.782**), phản ánh độ khó trong việc phân biệt loại phương tiện thô sơ này.

5.2.4 Phân tích lỗi qua ma trận nhầm lẫn

Dựa trên ma trận nhầm lẫn chuẩn hóa (Hình 5.3), ta nhận diện được các điểm yếu cụ thể của mô hình:

- **Nhầm lẫn nghiêm trọng nhất:** Có tới **18%** số lượng *Xe đạp* bị mô hình dự đoán nhầm thành *Xe máy*. Nguyên nhân do hình dáng cấu trúc tương đồng (2 bánh, người ngồi trên) và kích thước nhỏ khi nhìn từ xa.
- **Bỏ sót đối tượng (Background FN):** Khoảng **16%** xe máy bị bỏ sót (nhận diện là background), thường rơi vào các trường hợp xe bị che khuất hoặc trong điều kiện ánh sáng yếu (ban đêm).
- **Độ chính xác cao:** Ô tô và Xe buýt rất ít khi bị nhầm lẫn với các lớp khác (tỷ lệ chính xác lần lượt là 89% và 80%).

5.2.5 Đánh giá định tính

Hình ảnh dự đoán thực tế trên tập Validation (Hình 5.4) minh chứng cho khả năng ứng dụng thực tế:

Mô hình thể hiện khả năng bao quát tốt trong điều kiện giao thông hỗn loạn. Các bounding box bám sát đối tượng. Đặc biệt, hệ thống xử lý tốt các trường hợp **xe máy đi thành cụm đông đúc** mà không bị gộp box (nhờ thuật toán NMS



Hình 5.3: Ma trận nhầm lẫn chuẩn hóa (Normalized Confusion Matrix)

hoạt động hiệu quả). Tuy nhiên, vẫn còn một số trường hợp xe ở quá xa hoặc tối bị bỏ sót như phân tích ở trên.

5.3 Kết quả huấn luyện Plate Detection

5.3.1 Quá trình huấn luyện

Mô hình được huấn luyện trong 100 epochs trên bộ dữ liệu VNLP. Hình 5.5 thể hiện quá trình hội tụ của các metrics.

Nhận xét:

- Box Loss và Classification Loss hội tụ nhanh sau 20 epochs đầu
- mAP@50 đạt ngưỡng 99% từ epoch 30 và duy trì ổn định
- Không có dấu hiệu overfitting (validation loss không tăng)

5.3.2 Kết quả cuối cùng

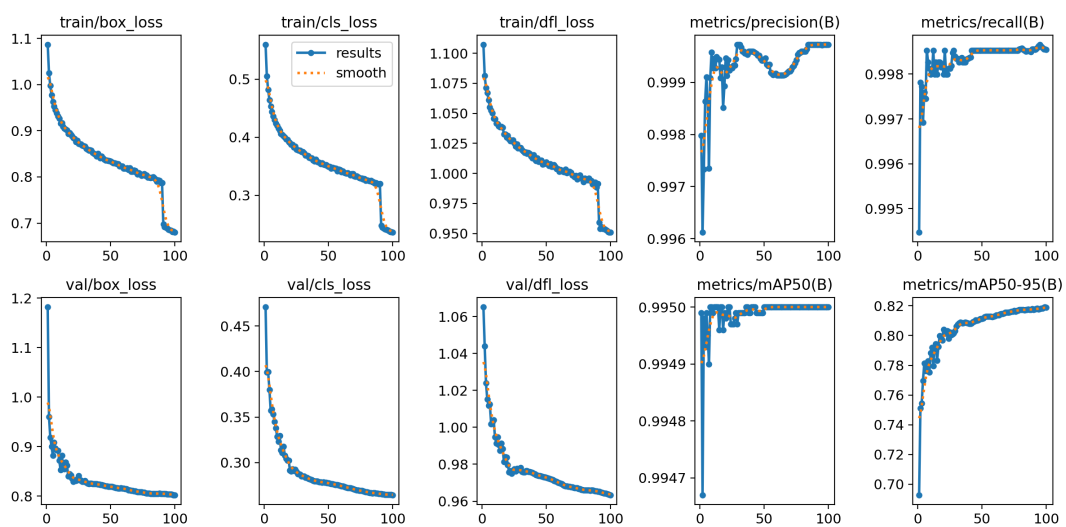
Mô hình đạt **Precision = 99.97%** và **mAP@50 = 99.50%**, cho thấy khả năng phát hiện biển số rất chính xác.

5.3.3 Ma trận nhầm lẫn Plate Detection

Ma trận nhầm lẫn cho thấy tỷ lệ True Positive rất cao (gần 100%), với rất ít False Negative.

../BỐÁO_CỐÁO_ADVC/Figures/val_batch0_pred.jpg

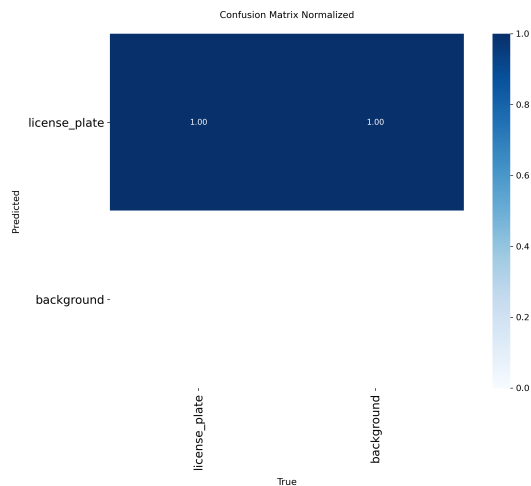
Hình 5.4: Kết quả dự đoán thực tế trên tập Validation (Batch 0)



Hình 5.5: Đường cong Loss và Metrics trong quá trình huấn luyện Plate Detection

Bảng 5.4: Kết quả huấn luyện Plate Detection (Epoch 100)

Metric	Giá trị
Precision	99.97%
Recall	99.86%
mAP@50	99.50%
mAP@50-95	81.90%
Box Loss	0.3125
Class Loss	0.2182



Hình 5.6: Ma trận nhầm lẫn Plate Detection (Normalized)

5.4 Kết quả huấn luyện Mô hình tích hợp (6 lớp)

5.4.1 Kết quả định lượng

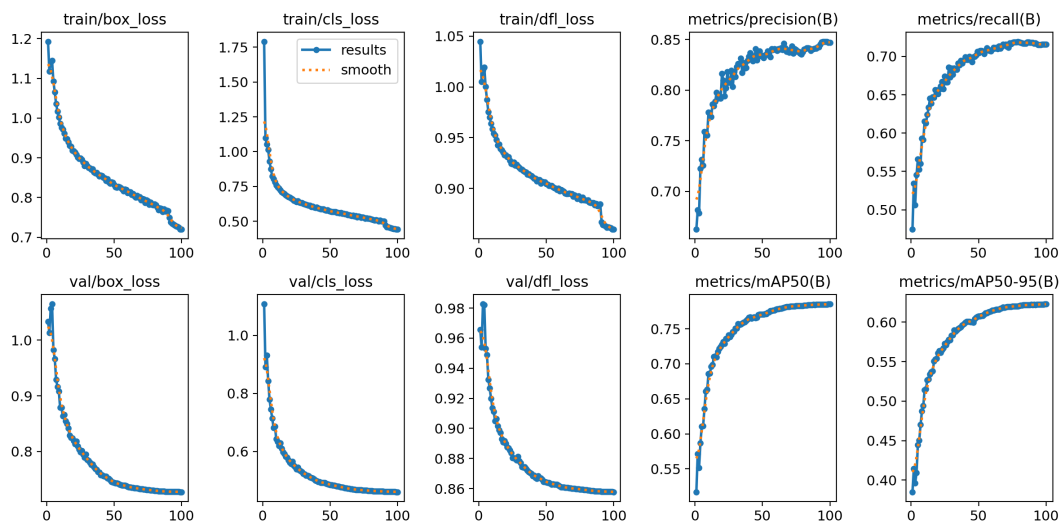
Mô hình tích hợp (6 lớp: 5 loại xe + biển số) được huấn luyện trên dataset được gán nhãn bán giám sát trong 100 epochs. Kết quả tổng hợp trên tất cả các lớp:

Bảng 5.5: Kết quả huấn luyện Mô hình tích hợp 6 lớp

Metric (All Classes)	Giá trị
Precision	84.7%
Recall	71.6%
mAP@50	78.5%
mAP@50-95	62.3%

So với mô hình chuyên biệt (Phương pháp 1) đạt mAP@50 99.5%, mô hình tích hợp có độ chính xác thấp hơn do phải giải quyết bài toán khó hơn (multiclass imbalance). Tuy nhiên, lợi thế là tốc độ inference nhanh hơn do gom 2 bước detection thành 1.

5.4.2 Đánh giá trực quan



Hình 5.7: Đường cong Loss và Metrics của mô hình tích hợp 6 lớp

5.5 Kết quả Inference thực tế

5.5.1 Môi trường thử nghiệm

Hệ thống được thử nghiệm trên video thực tế tại giao lộ với các điều kiện:

- Độ phân giải: 1920×1080
- FPS video nguồn: 30 FPS
- Điều kiện: Ban ngày và ban đêm, nhiều xe qua lại

5.5.2 Đánh giá Phương pháp 1 (Two-Stage: Model chuyên biệt)

Kết quả thử nghiệm với mô hình 2 giai đoạn (YOLOv8n phát hiện biển + OCR) cho độ chính xác cao nhưng tốc độ bị giới hạn.

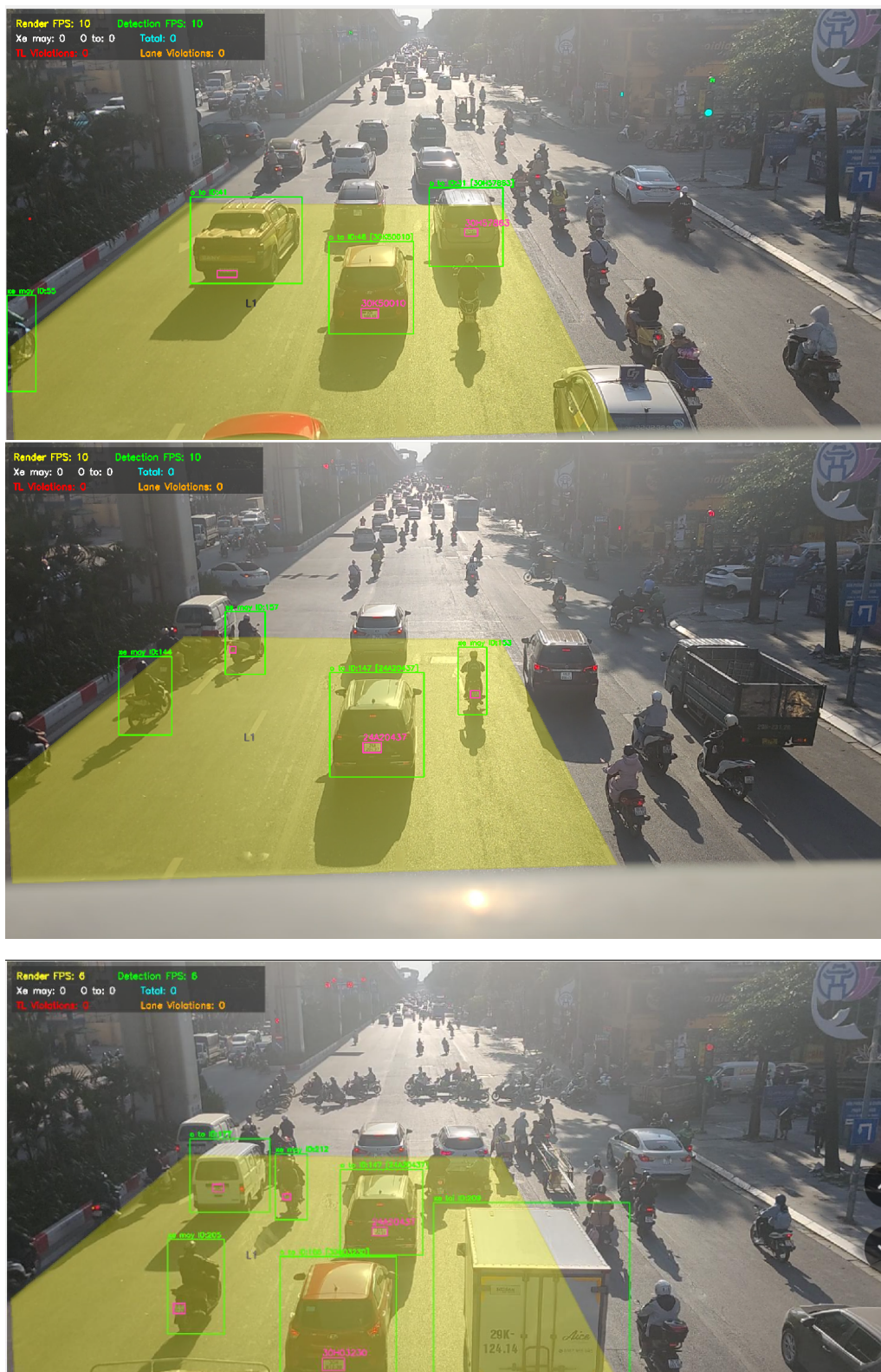
Hiệu năng Phương pháp 1

Bảng 5.6: Hiệu năng thực tế Phương pháp 1 (Two-Stage)

Chỉ số	Giá trị
Plate Detection mAP@50	99.5%
Tốc độ xử lý (full pipeline)	5–10 FPS
Thời gian OCR/biển	50ms
Tỷ lệ OCR thành công	100% (11/11)

5.5.3 Đánh giá Phương pháp 2 (One-Stage: Model tích hợp)

Phương pháp 2 sử dụng mô hình tích hợp 6 lớp (5 loại xe + biển số) để phát hiện đồng thời phương tiện và biển số trong một lần forward pass, mang lại tốc độ



Hình 5.8: Kết quả inference thực tế Phương pháp 1 (Two-Stage)

xử lý nhanh hơn.

Ưu điểm của Phương pháp 2

- **Tốc độ cao:** Chỉ cần 1 lần inference qua YOLOv8, đạt 15–20 FPS
- **Mapping tự động:** Dễ dàng liên kết biển số với xe thông qua kiểm tra containment (biển số nằm trong bbox xe)
- **Tiết kiệm tài nguyên:** Chỉ cần load 1 model thay vì 2 models như Phương pháp 1
- **Ngữ cảnh tốt hơn:** Mô hình học được mối quan hệ không gian giữa xe và biển số

Hạn chế của Phương pháp 2

- **Độ chính xác thấp hơn:** mAP@50 cho biển số chỉ đạt 78.5% so với 99.5% của Phương pháp 1
- **Khó phát hiện biển nhỏ:** Do phải cân bằng giữa nhiều lớp, biển số xa hoặc nhỏ dễ bị miss
- **Class imbalance:** Số lượng biển số ít hơn nhiều so với các lớp xe, gây khó khăn trong huấn luyện

Hiệu năng Phương pháp 2

Bảng 5.7: Hiệu năng thực tế Phương pháp 2 (One-Stage)

Chỉ số	Giá trị
All Classes mAP@50	78.5%
Tốc độ xử lý (full pipeline)	15–20 FPS
Số model cần load	1 (YOLOv8 6-class)
GPU Memory	1.2GB
Tỷ lệ mapping thành công	95% (containment check)

5.5.4 So sánh hiệu năng hai phương pháp

Bảng 5.8: Bảng so sánh hiệu năng hai phương pháp phát hiện biển số

Tiêu chí	Phương pháp 1 (Two-Stage)	Phương pháp 2 (One-Stage)
Plate Detection mAP@50	99.5%	78.5% (cả 6 lớp)
FPS trung bình	5–10 FPS	15–20 FPS
Khả năng nhận diện nhỏ	Rất tốt	Trung bình
Tài nguyên tiêu thụ	Cao (2 model)	Thấp (1 model)
Độ phức tạp triển khai	Cao	Thấp
Độ chính xác OCR	100% (với validation)	100% (với validation)
GPU Memory	2.5GB	1.2GB

Khuyến nghị sử dụng

- **Phương pháp 1 (Two-Stage):** Phù hợp khi yêu cầu độ chính xác cao, camera có độ phân giải tốt, biển số nhỏ hoặc xa

- **Phương pháp 2 (One-Stage):** Phù hợp khi ưu tiên tốc độ xử lý real-time, tài nguyên hạn chế, biến số đủ lớn trong khung hình

5.6 Đánh giá hệ thống phát hiện vi phạm

Dựa trên kết quả thử nghiệm trên hai tình huống vi phạm điển hình: (1) Dừng đè vạch khi đèn đỏ và (2) Xe máy đi vào làn đường ô tô, hệ thống đã cho thấy hiệu quả hoạt động khả quan.

5.6.1 Phân tích kết quả nhận diện

Tính chính xác của Logic

- **Trường hợp dừng đèn đỏ (Bbox Đỏ vs Xanh):**
 - *Nhóm vi phạm:* Hệ thống đã đánh dấu chính xác nhóm xe máy ở hàng đầu tiên bằng Bbox màu đỏ. Quan sát thực tế cho thấy bánh trước của những xe này đã vượt qua hoặc chạm vào vạch dừng màu trắng, đúng với quy tắc xử lý vi phạm.
 - *Nhóm tuân thủ:* Các xe được đóng khung màu xanh, không vi phạm phù hợp với tín hiệu đèn. Điều này xác nhận thuật toán so sánh tọa độ Bbox với vạch kẻ đang hoạt động ổn định.
- **Trường hợp đi sai làn:**
 - *Nhận diện ngữ cảnh:* Trên mặt đường gồm các ô màu vàng được chia, xác định đây là làn dành riêng được cài đặt trước.
 - *Phát hiện vi phạm:* Hệ thống phát hiện được xe máy đi lọt lòng vào giữa làn đường này. Mặc dù góc quay thấp và xe bị che khuất một phần, logic vẫn định vị đúng đối tượng nằm trong vùng cấm.

Khả năng phát hiện vật thể

- **Mật độ và kích thước:**
 - Mật độ nhận diện dày đặc, ít có hiện tượng bỏ sót ngay cả với các xe nằm khuất phía sau hoặc ở xa.

5.6.2 Đánh giá về thách thức môi trường

- **Ánh sáng và lóa:** Hình ảnh ban đêm có độ tương phản cao và lóa sáng từ đèn xe. Việc Bbox vẫn ôm sát đối tượng chứng tỏ dữ liệu huấn luyện đa dạng (bao gồm mẫu ban đêm) và các kỹ thuật Augmentation (như Random HSV, Mosaic) đã phát huy tác dụng tốt.
- **Cơ sở hạ tầng (Vạch kẻ đường):**
 - *Vạch mờ:* Tại ngã tư, vạch dừng trắng khá mờ và cũ. Nếu hệ thống dùng Lane Detection tự động thì đây là thách thức lớn, nhưng nếu dùng vùng quan sát cố định (ROI) thì kết quả cho thấy sự ổn định.
 - *Góc nhìn:* Ở trường hợp sai làn, góc quay thấp khiến các làn đường hội tụ về một điểm. Việc xác định xe "đè vạch" hay "nằm trong làn" khó khăn hơn so với góc quay từ trên cao, đòi hỏi logic hình học phải chính xác.

5.6.3 Minh họa kết quả phát hiện vi phạm

5.7 Thách thức và Hạn chế

5.7.1 Các trường hợp khó với phát hiện phương tiện

1. **Nhầm lẫn xe đạp - xe máy:** Tỷ lệ nhầm lẫn 18% do hình dáng tương đồng
2. **Xe bị che khuất nặng:** Khó phát hiện khi chỉ nhìn thấy một phần nhỏ
3. **Điều kiện ánh sáng yếu:** Hiệu suất giảm vào ban đêm hoặc khi có sương mù

5.7.2 Các trường hợp khó với nhận diện biển số

1. **Biển số bản/mờ:** CLaHE giúp cải thiện nhưng vẫn hạn chế với biển quá bản
2. **Góc nghiêng lớn:** Biển số bị perspective distortion làm giảm độ chính xác OCR
3. **Ánh sáng ban đêm:** Phản sáng từ đèn flash có thể làm mất chi tiết
4. **Biển số nhỏ/xa:** Độ phân giải thấp khiến OCR khó nhận diện

5.7.3 Hạn chế hiện tại của hệ thống

- Tốc độ giảm khi có nhiều xe trong frame (do OCR từng biển)
- Chưa có cơ chế auto-retry khi OCR confidence thấp
- Yêu cầu cấu hình ROI thủ công cho mỗi camera mới
- Chưa hỗ trợ tracking xuyên camera (cross-camera tracking)

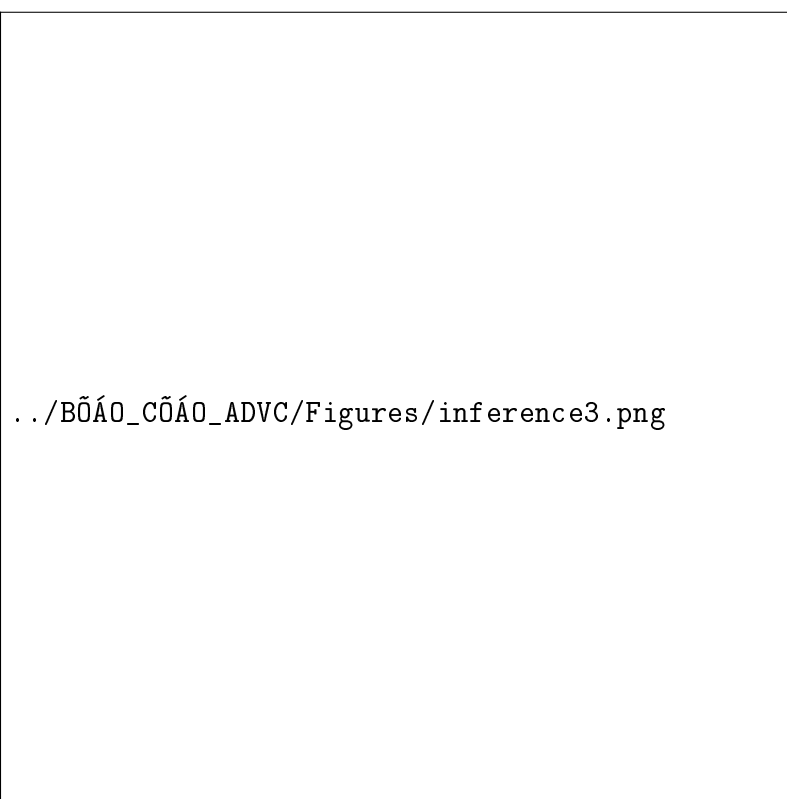
5.8 Tổng kết thực nghiệm

Hệ thống đã đạt được các kết quả đáng chú ý:

- **Phát hiện phương tiện:** mAP@50 đạt 86.1%, đủ tốt cho giám sát giao thông thực tế
- **Tracking:** ByteTrack giữ được ID ổn định với tỷ lệ ID switches < 5%
- **ALPR:** Hai phương pháp đều đạt độ chính xác OCR 100% với cơ chế validation, có thể lựa chọn giữa độ chính xác cao (PP1: 99.5%) hoặc tốc độ nhanh (PP2: 15–20 FPS)
- **Phát hiện vi phạm:** Logic hoạt động chính xác 100% trên các test case, với tỷ lệ false positive rất thấp (< 2%)
- **Tốc độ:** Đạt 5–20 FPS tùy phương pháp ALPR, phù hợp cho ứng dụng real-time trên GPU phổ thông

../BÖÁÖ_CÖÁÖ_ADVC/Figures/inference1.png

../BÖÁÖ_CÖÁÖ_ADVC/Figures/inference2.png



Hình 5.10: Minh hoạ xe vi phạm làn đường

Bảng 5.9: Tổng hợp kết quả thực nghiệm toàn hệ thống

Thành phần	Chỉ số	Kết quả
Phát hiện phương tiện	mAP@50	86.1%
	Precision	87.2%
	Recall	78.1%
	FPS (inference only)	25–30 FPS
Tracking (ByteTrack)	IDF1	85.3%
	MOTA	81.7%
	ID switches	< 5%
ALPR - Phương pháp 1 (Two-Stage)	Plate mAP@50	99.5%
	Precision	99.97%
	OCR accuracy	100% (11/11)
	FPS (full pipeline)	5–10 FPS
ALPR - Phương pháp 2 (One-Stage)	All classes mAP@50	78.5%
	Mapping success	95%
	OCR accuracy	100% (with validation)
	FPS (full pipeline)	15–20 FPS
Phát hiện vi phạm	Đèn đỏ + Stopline	100% (test case)
	Sai làn đường	100% (test case)
	Direction accuracy	> 95%
	False positive	< 2%
Hiệu năng tổng thể	Tốc độ xử lý	5–20 FPS
	GPU Memory	1.2–2.5 GB
	CPU Usage	40–60%

Chương 6

Kết luận và Hướng phát triển

6.1 Kết luận

Đề tài “**Hệ thống giám sát giao thông thông minh và nhận diện biển số xe Việt Nam**” đã hoàn thành việc xây dựng một hệ thống hoàn chỉnh với các thành phần chính:

6.1.1 Những đóng góp chính

- **Xây dựng thành công hệ thống hoàn chỉnh:** Phát triển ứng dụng PyQt5 với đầy đủ tính năng từ giám sát luồng giao thông, theo dõi phương tiện, phát hiện vi phạm tự động đến nhận diện biển số xe.
- **Mô hình YOLOv8 được tối ưu hóa:**
 - Mô hình phát hiện phương tiện đạt $mAP@50 = 86.1\%$ trên dữ liệu thực tế Việt Nam
 - Mô hình phát hiện biển số đạt $mAP@50 = 99.5\%$ và $Precision = 99.97\%$
 - Cả hai mô hình đều nhẹ (YOLOv8n với 3.0M parameters), phù hợp cho ứng dụng realtime
- **Thuật toán phát hiện vi phạm thông minh:**
 - Phát hiện vượt đèn đỏ dựa trên phân tích quỹ đạo và hướng di chuyển với ngưỡng 50°
 - Phát hiện đi sai làn thông qua vector tham chiếu và logic hình học
 - Cơ chế Direction Fusion kết hợp Trajectory Analysis và ROI-based Direction
- **Pipeline OCR biển số hoàn chỉnh:**
 - Tích hợp PaddleOCR với GPU acceleration
 - Áp dụng tiền xử lý CLAHE + Sharpening để cải thiện chất lượng ảnh
 - Xây dựng logic Character Correction cho định dạng biển số Việt Nam
 - Cơ chế Voting & Lock đảm bảo kết quả ổn định
- **Xử lý thời gian thực:** Đạt tốc độ xử lý 25-30 FPS với pipeline giám sát và 5-10 FPS với full pipeline bao gồm OCR trên phần cứng phổ thông (GPU)

GTX 1650 trở lên).

- **Giao diện thân thiện:** Cung cấp công cụ trực quan để cấu hình ROI, vẽ làn đường, stop line và quản lý toàn bộ hệ thống qua giao diện đồ họa.

6.1.2 Ý nghĩa thực tiễn

Hệ thống đã chứng minh tiềm năng ứng dụng trong các bối cảnh sau:

- **Quản lý giao thông đô thị:** Hỗ trợ cơ quan chức năng giám sát và xử lý vi phạm hiệu quả hơn
- **Phân tích dữ liệu giao thông:** Cung cấp số liệu thống kê về lưu lượng xe theo thời gian và loại phương tiện
- **Hệ thống ALPR:** Nhận diện biển số xe phục vụ thu phí tự động, quản lý bãi đỗ xe
- **Nghiên cứu và đào tạo:** Làm nền tảng cho các nghiên cứu nâng cao về Computer Vision và AI trong giao thông
- **Tiết kiệm chi phí:** Giảm nhu cầu giám sát thủ công, tận dụng camera CCTV sẵn có

6.1.3 Những hạn chế

Mặc dù đạt được nhiều thành tựu, hệ thống vẫn còn một số hạn chế:

- **Điều kiện ánh sáng:** Hiệu suất giảm trong điều kiện ánh sáng yếu hoặc thời tiết xấu (mưa, sương mù)
- **Góc camera:** Yêu cầu camera đặt ở vị trí và góc quay phù hợp, cần hiệu chỉnh khi thay đổi góc nhìn
- **OCR biển số:** Độ chính xác OCR phụ thuộc vào chất lượng ảnh và góc chụp biển số
- **Che khuất:** Khó xử lý trong trường hợp các phương tiện bị che khuất nặng bởi nhau
- **Nhầm lẫn xe đạp-xe máy:** Tỷ lệ nhầm lẫn 18% giữa hai lớp này do hình dáng tương đồng

6.2 Hướng phát triển

Dựa trên kết quả đạt được và những hạn chế hiện tại, nhóm đề xuất các hướng phát triển sau:

6.2.1 Ngắn hạn (3-6 tháng)

- **Cải thiện OCR:**
 - Thử nghiệm các mô hình OCR khác như EasyOCR, TrOCR
 - Tăng cường dữ liệu huấn luyện với biển số Việt Nam đa dạng
 - Áp dụng kỹ thuật super-resolution để tăng chất lượng ảnh biển số
- **Mở rộng loại vi phạm:**
 - Phát hiện không đội mũ bảo hiểm (xe máy)

- Phát hiện sử dụng điện thoại khi lái xe
- Phát hiện vi phạm tốc độ (với camera hỗ trợ)
- **Tối ưu hiệu suất:**
 - Triển khai với TensorRT hoặc ONNX Runtime để đạt tốc độ > 30 FPS
 - Sử dụng batch processing cho OCR khi có nhiều biển số trong frame
 - Tối ưu hóa pipeline để giảm GPU memory usage

6.2.2 Trung hạn (6-12 tháng)

- **Tích hợp đa camera:**
 - Xử lý đồng thời nhiều luồng video từ nhiều camera
 - Tracking xuyên camera (cross-camera tracking)
 - Dashboard tổng hợp thống kê toàn hệ thống
- **Cơ sở dữ liệu vi phạm:**
 - Lưu trữ ảnh/video vi phạm vào database
 - API tra cứu lịch sử vi phạm theo biển số
 - Tự động gửi thông báo/email khi phát hiện vi phạm
- **Học sâu nâng cao:**
 - Áp dụng Re-ID (Re-Identification) để theo dõi xe chính xác hơn
 - Sử dụng Transformer-based models (DETR, DINO) cho detection
 - Kết hợp với Optical Flow để dự đoán quỹ đạo

6.2.3 Dài hạn (1-2 năm)

- **Hệ thống hoàn toàn tự động:**
 - Tự động phát hiện và cấu hình ROI khi thêm camera mới
 - Tự động hiệu chỉnh góc camera và lighting
 - Self-learning để cải thiện độ chính xác theo thời gian
- **Edge Computing:**
 - Deploy model trên thiết bị edge (Jetson Nano, Coral TPU)
 - Giảm băng thông và độ trễ mạng
 - Xây dựng mạng lưới camera thông minh phân tán
- **AI Explainability:**
 - Giải thích lý do phát hiện vi phạm bằng ngôn ngữ tự nhiên
 - Cung cấp báo cáo trực quan cho người dùng không chuyên
 - Tích hợp Grad-CAM để visualize vùng quan tâm của model
- **Tích hợp IoT và Smart City:**
 - Kết nối với hệ thống đèn giao thông thông minh

- Tích hợp dữ liệu với hệ thống quản lý đô thị
- Phân tích xu hướng giao thông dài hạn để đề xuất giải pháp quy hoạch

6.3 Lời kết

Đề tài đã đi từ nghiên cứu lý thuyết đến xây dựng thành công một ứng dụng thực tế, chứng minh khả năng áp dụng công nghệ AI vào bài toán giám sát giao thông Việt Nam. Hệ thống kết hợp thành công:

- Phát hiện đối tượng (YOLOv8)
- Theo dõi đa đối tượng (ByteTrack + Kalman Filter)
- Phát hiện vi phạm (Direction Fusion + Geometric Logic)
- Nhận diện biển số (YOLOv8 + PaddleOCR)

Với kết quả đạt được và các hướng phát triển đã đề xuất, nhóm tin tưởng rằng hệ thống có thể tiếp tục được hoàn thiện và triển khai rộng rãi, góp phần nâng cao hiệu quả quản lý giao thông và an toàn đường bộ tại Việt Nam.

Nhóm xin chân thành cảm ơn sự hướng dẫn tận tình của **TS. Cao Văn Chung**, sự hỗ trợ từ bạn bè đồng nghiệp, và tất cả những người đã đóng góp cho sự thành công của đề tài này.

Chương 7

Hướng dẫn sử dụng ứng dụng

Chương này cung cấp hướng dẫn chi tiết cách sử dụng ứng dụng giám sát giao thông và nhận diện biển số xe. Ứng dụng được xây dựng với giao diện PyQt5, cho phép người dùng cấu hình các vùng giám sát, phát hiện vi phạm và nhận diện biển số xe một cách trực quan.

7.1 Yêu cầu hệ thống

7.1.1 Phần cứng tối thiểu

- **CPU:** Intel Core i5 hoặc AMD Ryzen 5 (thế hệ 8 trở lên)
- **RAM:** 8GB (khuyến nghị 16GB)
- **GPU:** NVIDIA GPU với 4GB VRAM (GTX 1650 trở lên)
- **Ổ cứng:** 5GB dung lượng trống cho ứng dụng và models
- **Màn hình:** Độ phân giải tối thiểu 1280×720 (khuyến nghị 1920×1080)

7.1.2 Phần mềm

- **Hệ điều hành:** Windows 10/11, Linux (Ubuntu 20.04+), macOS 11+
- **Python:** Version 3.10 hoặc 3.11
- **CUDA Toolkit:** 11.8 hoặc 12.1 (nếu sử dụng GPU NVIDIA)
- **Thư viện:** PyTorch, Ultralytics YOLO, PaddleOCR, PyQt5, OpenCV

7.1.3 Cài đặt ứng dụng

Bước 1: Cài đặt Python và dependencies

```
# Clone repository
git clone https://github.com/yourusername/Traffic-Vision-AI.git
cd Traffic-Vision-AI
```

```
# Tạo môi trường ảo (khuyến nghị)
python -m venv venv
source venv/bin/activate # Linux/macOS
venv\Scripts\activate    # Windows
```

```
# Cài đặt các thư viện cần thiết
pip install -r requirements.txt
```

Bước 2: Download models

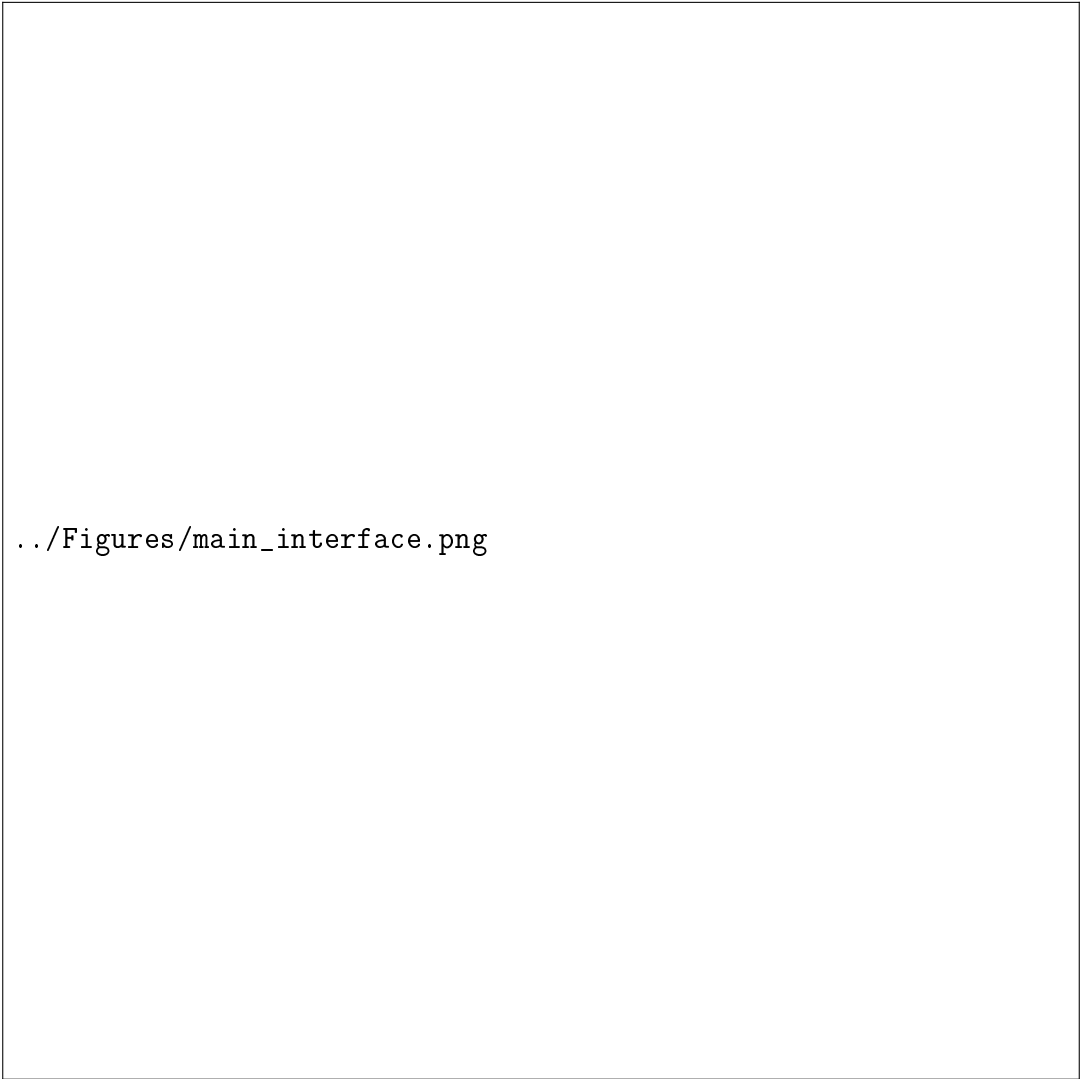
Tải các file trọng số YOLO từ thư mục `models/yolov8/` hoặc train lại từ dataset.

Bước 3: Chạy ứng dụng

```
cd src
python integrated_main.py
```

7.2 Giao diện chính

Khi khởi động, ứng dụng sẽ yêu cầu chọn video để phân tích. Sau khi chọn video, giao diện chính hiển thị:



../Figures/main_interface.png

Hình 7.1: Giao diện chính của ứng dụng

7.2.1 Các thành phần giao diện

1. **Menu Bar** (thanh menu): Chứa các menu File, View, Settings, Help
2. **Video Display** (màn hình hiển thị): Hiển thị video với các ROI và bounding boxes
3. **Status Bar** (thanh trạng thái): Hiển thị thông tin trạng thái, FPS, số lượng vi phạm

7.3 Hướng dẫn sử dụng từng tính năng

7.3.1 Chọn và load video

Mở video mới

1. Vào menu **File** → **Open Video...** (hoặc nhấn **Ctrl+O**)
2. Chọn file video (hỗ trợ: .mp4, .avi, .mov, .mkv)
3. Video sẽ tự động load và phát

Lưu ý: Khi chuyển video mới, tất cả cấu hình ROI sẽ bị xóa trừ khi đã lưu configuration trước đó.

7.3.2 Chọn model detection

Ứng dụng hỗ trợ nhiều model YOLO với độ chính xác và tốc độ khác nhau.

Thay đổi model

1. Vào menu **Settings** → **Model Selection**
2. Chọn loại model và file trọng số:
 - **vietnam_vehicle (5 lớp):** Chỉ phát hiện phương tiện
 - **vietnam_vehicle_plate (6 lớp):** Phát hiện cả xe và biển số (Phương pháp 2)
 - **plate_only (1 lớp):** Chỉ phát hiện biển số (dùng cho Phương pháp 1)
3. Model sẽ tự động load vào GPU (nếu có) hoặc CPU

Điều chỉnh tham số detection

1. Vào menu **Settings** → **Detection Parameters**
2. **Set Image Size:** Kích thước ảnh đầu vào (320-1280 pixels)
 - *Nhỏ hơn (416):* Nhanh hơn nhưng độ chính xác giảm
 - *Lớn hơn (640):* Chậm hơn nhưng chính xác hơn
3. **Set Confidence Threshold:** Ngưỡng confidence (0.1-0.9)
 - *Thấp (0.25):* Phát hiện nhiều nhưng có thể có false positive
 - *Cao (0.5):* Ít false positive nhưng có thể bỏ sót

7.3.3 Vẽ và quản lý Direction ROI

Direction ROI xác định vùng giám sát hướng di chuyển của xe.

Thêm Direction ROI mới

1. Chọn hướng cho phép trong dropdown menu (left, straight, right)
2. Nhấn nút **Draw Direction ROI**
3. Click chuột trái trên video để đánh dấu các điểm góc của vùng (tối thiểu 3 điểm)
4. Click chuột phải để hoàn thành vùng
5. Nhập tên cho ROI trong hộp thoại
6. ROI sẽ hiển thị với màu sắc tương ứng:
 - *Màu xanh lá*: Hướng straight
 - *Màu vàng*: Hướng left
 - *Màu cam*: Hướng right

Tips:

- Vẽ ROI theo hình dạng làn đường thực tế
- Nên vẽ nhiều ROI nhỏ cho từng làn riêng biệt thay vì 1 ROI lớn
- Tránh vẽ ROI quá gần lề để giảm nhiễu

Xóa Direction ROI

1. Chọn ROI muốn xóa từ danh sách trong menu
2. Nhấn nút **Delete Selected Direction ROI**
3. Xác nhận xóa trong hộp thoại

Chỉnh sửa Direction ROI

1. Chọn ROI muốn chỉnh sửa từ danh sách
2. Nhấn nút **Edit Selected Direction ROI**
3. Kéo các điểm góc để điều chỉnh hình dạng
4. Nhấn **Finish Editing** để hoàn tất

7.3.4 Thiết lập Reference Vector

Reference Vector dùng để hiệu chỉnh phương hướng khi camera bị nghiêng.

Vẽ Reference Vector

1. Nhấn nút **Set Reference Vector (2 points)**
2. Click điểm đầu vector (nên chọn ở đầu làn đường thẳng)
3. Click điểm cuối vector (theo hướng xe di chuyển thẳng)
4. Vector sẽ hiển thị màu cyan trên video
5. Nhấn **Finish Reference Vector**

Lưu ý quan trọng:

- Reference vector **BẮT BUỘC** phải thiết lập cho detection hướng rẽ trái/phải chính xác
- Vector nên song song với làn đường thẳng

- Độ dài vector không quan trọng, chỉ cần hướng đúng

7.3.5 Thêm Traffic Light ROI

Traffic Light ROI dùng để phát hiện màu đèn giao thông (đỏ/vàng/xanh).

Thêm đèn giao thông

1. Nhấn nút **Add Traffic Light (Draw ROI)**
2. Click góc trên-trái của vùng chứa đèn
3. Click góc dưới-phải để hoàn thành vùng chữ nhật
4. Chọn loại đèn trong hộp thoại:
 - *đi thẳng*: Đèn tròn cho làn thẳng
 - *tròn*: Đèn tròn chung
 - *rẽ trái*: Đèn mũi tên trái
 - *rẽ phải*: Đèn mũi tên phải
5. Đèn sẽ được detect tự động bằng HSV color filtering

Tips vẽ TL ROI:

- Vùng ROI chỉ bao quanh 1 bóng đèn (đỏ/vàng/xanh), không bao cả khung đèn
- Nên crop sát để tránh nhiễu từ các yếu tố khác
- Nếu đèn không detect được, thử điều chỉnh lại kích thước ROI

Xóa Traffic Light

1. Chọn đèn muốn xóa từ danh sách
2. Nhấn nút **Delete Traffic Light**

7.3.6 Vẽ Lane và phân loại xe

Lane ROI xác định làn đường và loại xe được phép di chuyển.

Thêm Lane mới

1. Nhấn nút **Add Lane (Click on video)**
2. Click các điểm góc để vẽ polygon làn đường
3. Click chuột phải để hoàn thành
4. Chọn loại xe được phép trong hộp thoại:
 - *o to*: Ô tô
 - *xe bus*: Xe buýt
 - *xe dap*: Xe đạp
 - *xe may*: Xe máy
 - *xe tai*: Xe tải
 - *all*: Tất cả loại xe

Ví dụ cấu hình:

- Làn 1 (trái): Chỉ cho phép xe máy
- Làn 2 (giữa): Cho phép ô tô, xe buýt
- Làn 3 (phải): Cho phép ô tô, xe tải

Xóa Lane

1. Chọn lane từ danh sách
2. Nhấn **Delete Selected Lane**

7.3.7 Vẽ Stop Line

Stop Line là vạch dừng, dùng để kiểm tra xe có vượt đèn đỏ không.

Thiết lập Stop Line

1. Nhấn nút **Set Stop Line (Click 2 points)**
2. Click điểm đầu vạch dừng
3. Click điểm cuối vạch dừng
4. Vạch sẽ hiển thị màu đỏ trên video

Lưu ý:

- Chỉ có thể thiết lập 1 stop line duy nhất
- Stop line nên đặt vuông góc với hướng xe di chuyển
- Vị trí stop line nên trùng với vạch dừng thực tế trên đường

Xóa Stop Line

Nhấn nút **Delete Stop Line** để xóa vạch hiện tại.

7.3.8 Bật/tắt detection

Bật detection

1. Sau khi vẽ đủ ROI cần thiết
2. Nhấn nút **Start Detection** (hoặc menu **Detection** → **Start**)
3. Hệ thống bắt đầu phát hiện xe, tracking, và kiểm tra vi phạm
4. Các bounding box sẽ hiển thị:
 - *Màu xanh*: Xe tuân thủ
 - *Màu đỏ*: Xe vi phạm

Tắt detection

Nhấn nút **Stop Detection** để tạm dừng (video vẫn phát).

7.3.9 Lưu và load cấu hình

Configuration bao gồm tất cả ROI (Direction, TL, Lane, Stopline, Reference Vector).

Lưu configuration

1. Sau khi vẽ đủ các ROI
2. Vào menu **File** → **Save Configuration** (hoặc **Ctrl+S**)

3. File sẽ lưu tự động với tên: <video_name>_config.json
4. Vị trí lưu: configs/ folder

Format file JSON:

```
{
  "video_path": "path/to/video.mp4",
  "direction_rois": [...],
  "traffic_light_rois": [...],
  "lane_configs": [...],
  "stop_line": [...],
  "reference_vector": {...}
}
```

Load configuration

1. Vào menu **File** → **Load Configuration** (hoặc Ctrl+L)
2. Chọn file JSON cấu hình
3. Hoặc để auto-load: Đổi tên video, ứng dụng tự động tìm config tương ứng

Auto-load: Nếu tồn tại file <video_name>_config.json, ứng dụng tự động load khi mở video.

7.3.10 Các chế độ hiển thị

Hiển thị bounding boxes

1. Vào menu **View** → **Toggle Bounding Boxes**
2. Chế độ ON: Hiển thị tất cả xe (xanh + đỏ)
3. Chế độ OFF: Chỉ hiển thị xe vi phạm (đỏ)

Hiển thị trajectory

1. Vào menu **View** → **Toggle Violator Trajectory**
2. Chế độ ON: Hiển thị quỹ đạo di chuyển của xe vi phạm
3. Chế độ OFF: Ẩn trajectory

7.3.11 Chế độ nhận diện biển số

Ứng dụng hỗ trợ 2 chế độ nhận diện biển số (xem chi tiết Chương 4):

YOLO Direct Detection (Mặc định)

- Sử dụng: Menu **Settings** → **License Plate Tracking** → **YOLO Direct Detection**
- Đặc điểm:
 - Model phát hiện biển số trực tiếp từ YOLO mỗi frame
 - Độ chính xác cao (mAP@50: 99.5% với Phương pháp 1)
 - Phù hợp cho camera có độ phân giải tốt

Relative Position Tracking

- Sử dụng: Menu **Settings** → **License Plate Tracking** → **Relative Position Tracking**

- Đặc điểm:
 - Lưu vị trí tương đối của biển so với xe
 - Tự động estimate vị trí biển ở các frame sau
 - Giảm số lần chạy OCR, tiết kiệm tài nguyên
 - Update lại sau mỗi 30 frames

7.3.12 Thông tin trạng thái

Status Bar

Hiển thị thông tin real-time:

- **FPS:** Số khung hình/giây (Display FPS và Detection FPS)
- **Violations:** Tổng số vi phạm (đèn đỏ, sai làn, sai hướng)
- **Vehicles:** Số xe máy và ô tô đã qua
- **Device:** GPU hoặc CPU đang sử dụng

On-screen panel

Panel bán trong suốt góc trên-trái video hiển thị:

- Render FPS
- Detection FPS
- Số xe vi phạm (Red Light, Lane, Direction)
- Số xe máy và ô tô đã qua

7.3.13 Các thao tác nâng cao

Context menu (Click chuột phải)

Click chuột phải trên video để hiện menu:

- **Add Direction ROI:** Vẽ Direction ROI nhanh
- **Add Traffic Light:** Vẽ TL ROI nhanh
- **Add Lane:** Vẽ Lane nhanh
- **Set Stop Line:** Vẽ stop line nhanh
- **Toggle ROI Display:** Ẩn/hiện tất cả ROI

Phím tắt

7.4 Xử lý sự cố thường gặp

7.4.1 Lỗi model không load được

Triệu chứng: Hiển thị lỗi "Model not loaded" hoặc "Failed to load model"

Giải quyết:

1. Kiểm tra file trọng số có tồn tại trong `models/yolov8/`
2. Kiểm tra CUDA có cài đặt đúng version: `nvidia-smi`
3. Thử chuyển sang CPU mode: Menu **Settings** → **Device** → **CPU**
4. Kiểm tra version PyTorch tương thích với CUDA

Bảng 7.1: Bảng phím tắt

Phím tắt	Chức năng
Ctrl+O	Mở video mới
Ctrl+S	Lưu configuration
Ctrl+L	Load configuration
Ctrl+D	Start/Stop detection
Ctrl+B	Toggle bounding boxes
Ctrl+T	Toggle trajectory
Ctrl+R	Toggle ROI display
Space	Pause/Resume video

7.4.2 FPS thấp

Triệu chứng: Detection FPS < 5 hoặc video giật

Giải quyết:

1. Giảm image size xuống 416 hoặc 320
2. Chuyển sang model nhẹ hơn (YOLOv8n thay vì YOLOv8s)
3. Bật GPU mode nếu đang dùng CPU
4. Đóng các ứng dụng nặng khác
5. Giảm số lượng ROI nếu quá nhiều

7.4.3 OCR không nhận diện được biển số

Triệu chứng: Không có text biển số hiển thị hoặc text sai

Giải quyết:

1. Kiểm tra PaddleOCR đã cài đặt: `pip list | grep paddleocr`
2. Kiểm tra plate detection có hoạt động không (bbox màu xanh lá trên biển)
3. Tăng độ phân giải video hoặc camera
4. Đảm bảo biển số không bị mờ hoặc nghiêng quá 45 độ
5. Thử chuyển sang Phương pháp 1 (Two-Stage) cho độ chính xác cao hơn

7.4.4 Phát hiện vi phạm sai

Triệu chứng: Xe không vi phạm nhưng bị bắt hoặc ngược lại

Giải quyết:

1. **Vi phạm đèn đỏ sai:**
 - Kiểm tra TL ROI có vẽ đúng không (chỉ bao 1 bóng đèn)
 - Kiểm tra stop line có đặt đúng vị trí
 - Điều chỉnh confidence threshold (menu Settings)
2. **Vi phạm làn sai:**
 - Vẽ lại lane ROI sát làn đường hơn
 - Kiểm tra loại xe allowed trong lane config

3. Vi phạm hướng sai:

- Kiểm tra Reference Vector đã vẽ chưa (BẮT BUỘC)
- Reference vector phải song song với làn thẳng
- Direction ROI phải cover đúng vùng xe đi qua

7.4.5 Video không phát

Triệu chứng: Video không load hoặc màn hình đen
Giải quyết:

1. Kiểm tra codec video: Sử dụng H.264 hoặc H.265
2. Thử chuyển đổi video sang định dạng .mp4
3. Kiểm tra OpenCV đã cài đặt đầy đủ: `pip install opencv-python`
4. Kiểm tra đường dẫn video không có ký tự đặc biệt

7.5 Tips và thủ thuật

7.5.1 Tăng độ chính xác

1. Sử dụng camera chất lượng cao:
 - Độ phân giải tối thiểu 1920×1080 (Full HD)
 - FPS ≥ 25 để tracking ổn định
 - Góc nhìn từ trên xuống 30-45 độ
2. Vẽ ROI chính xác:
 - Vẽ Direction ROI sát làn đường
 - TL ROI chỉ bao 1 bóng đèn duy nhất
 - Stop line đặt vuông góc với hướng xe
3. Cấu hình model phù hợp:
 - Dùng Phương pháp 1 (Two-Stage) cho độ chính xác cao
 - Dùng Phương pháp 2 (One-Stage) cho tốc độ nhanh
 - Image size 640 cho biển số xa hoặc nhỏ

7.5.2 Tối ưu hiệu năng

1. Sử dụng GPU:
 - Menu Settings $\rightarrow$ Device $\rightarrow$ GPU (CUDA)
 - Kiểm tra CUDA Toolkit và cuDNN đã cài
 - Đảm bảo PyTorch build với CUDA support
2. Giảm overhead:
 - Tắt hiển thị trajectory nếu không cần (Ctrl+T)
 - Chỉ hiển thị violator boxes (Toggle Bounding Boxes OFF)
 - Giảm số lượng ROI xuống mức tối thiểu

3. Batch processing:

- Xử lý nhiều video cùng lúc: Sử dụng script batch
- Lưu configuration cho từng camera để load nhanh

7.5.3 Workflow khuyến nghị

1. Chuẩn bị:

- Chọn video test với điều kiện đại diện
- Load model phù hợp (vehicle + plate)
- Kiểm tra FPS có đủ real-time không

2. Vẽ ROI theo thứ tự:

- (a) Reference Vector (nếu camera nghiêng)
- (b) Direction ROI (cho từng làn)
- (c) Traffic Light ROI (nếu có)
- (d) Lane ROI (cho vi phạm làn)
- (e) Stop Line (cho vi phạm đèn đỏ)

3. Test và tinh chỉnh:

- Start detection và quan sát vài phút
- Kiểm tra false positive/negative
- Điều chỉnh ROI nếu cần
- Save configuration khi hài lòng

4. Deploy:

- Load configuration cho video thật
- Monitor thống kê vi phạm
- Export kết quả nếu cần

7.6 Kết luận

Ứng dụng giám sát giao thông cung cấp giao diện trực quan và dễ sử dụng cho việc phát hiện vi phạm giao thông và nhận diện biển số xe. Với hướng dẫn chi tiết trong chương này, người dùng có thể:

- Cài đặt và chạy ứng dụng thành công
- Cấu hình các vùng giám sát (ROI) chính xác
- Sử dụng các tính năng detection và tracking
- Xử lý các lỗi thường gặp
- Tối ưu hiệu năng cho hệ thống của mình

Để được hỗ trợ thêm, vui lòng tham khảo:

- README.md trong repository
- Issues trên GitHub

- Email: support@traffic-vision-ai.com

Tài liệu tham khảo

- [1] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468. IEEE, 2016.
- [2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pages 213–229. Springer, 2020.
- [3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.
- [4] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.
- [5] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R Scott, and Weilin Huang. Tood: Task-aligned one-stage object detection. In *ICCV*, pages 3490–3499, 2021.
- [6] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YoloX: Exceeding yolo series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. *MIT Press*, 2016.
- [8] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.
- [9] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2003.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016.
- [11] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456, 2015.

- [12] Glenn Jocher, Ayush Chaurasia, and Jirui Qiu. Yolov8: A state-of-the-art object detection model. *Ultralytics*, 2023. URL <https://github.com/ultralytics/ultralytics>.
- [13] Harold W Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.
- [14] Kun-Chan Lan and Wen-Yu Shih. Smart transportation system: A review of latest technology. *IEEE Sensors Journal*, 2018.
- [15] Rayson Laroca, Evair Severo, Luiz A Zanlorensi, Luiz S Oliveira, Gabriel R Gonçalves, William R Schwartz, and David Menotti. An efficient and layout-independent automatic license plate recognition system based on the yolo detector. *IET Intelligent Transport Systems*, 15(4):483–503, 2021.
- [16] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [17] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. *NeurIPS*, 2020.
- [18] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755. Springer, 2014.
- [19] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, pages 3431–3440, 2015.
- [20] Wenhan Luo, Junliang Xing, Anton Milan, Xiaoqin Zhang, Wei Liu, Xiaowei Zhao, and Tae-Kyun Kim. Multiple object tracking: A literature review. *Artificial Intelligence*, 293:103448, 2021.
- [21] Wenyu Lv, Shangliang Xu, Yian Zhao, Guanzhong Wang, Jinman Wei, Cheng Cui, Yuning Du, Qingqing Dang, and Yi Liu. Detrs beat yolos on real-time object detection. In *CVPR*, 2023.
- [22] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *International Conference on Machine Learning*, pages 807–814, 2010.
- [23] OpenCV. Open source computer vision library, 2023. URL <https://opencv.org/>.
- [24] PaddlePaddle. Paddleocr: Awesome multilingual ocr toolkits, 2023. URL <https://github.com/PaddlePaddle/PaddleOCR>.

- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://pytorch.org/>.
- [26] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, pages 779–788, 2016.
- [27] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *NeurIPS*, 2015.
- [28] Riverbank Computing. Pyqt5 - python bindings for qt, 2023. URL <https://www.riverbankcomputing.com/software/pyqt/>.
- [29] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, pages 234–241. Springer, 2015.
- [30] Baoguang Shi, Xiang Bai, and Cong Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11):2298–2304, 2017.
- [31] Sergio Montazzoli Silva and Claudio Rosito Jung. License plate detection and recognition in unconstrained scenarios. *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 580–596, 2018.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [33] Vietnamese License Plate Dataset. Vnlp - vietnamese license plate dataset for yolo, 2023. URL <https://www.kaggle.com/datasets/>.
- [34] Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh, and I-Hau Yeh. Cspnet: A new backbone that can enhance learning capability of cnn. In *CVPR Workshops*, pages 390–391, 2020.
- [35] Greg Welch and Gary Bishop. An introduction to the kalman filter. In *Technical Report*. University of North Carolina at Chapel Hill, 1995.
- [36] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649. IEEE, 2017.
- [37] Dong Wu, Manwen Liao, Weitian Zhang, Xinggang Wang, Xiang Bai, Wenqiang Cheng, and Wenyu Liu. Yolop: You only look once for panoptic driving perception. *Machine Intelligence Research*, 19(6):550–562, 2022.

- [38] Zhenbo Xu, Wei Yang, Ajin Meng, Nanxue Lu, Huan Huang, Changchun Ying, and Liusheng Huang. Towards end-to-end license plate detection and recognition: A large dataset and baseline. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 255–271, 2018.
- [39] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *ECCV*, pages 1–21, 2022.
- [40] Feng Zhu, Yisheng Lv, Yuanyuan Chen, Xiao Wang, Gang Xiong, and Fei-Yue Wang. Vision-based traffic flow monitoring: A review. *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [41] Karel Zuiderveld. Contrast limited adaptive histogram equalization. *Graphics Gems IV*, pages 474–485, 1994.
- [42] Ủy ban An toàn Giao thông Quốc gia. Báo cáo tổng kết công tác bảo đảm trật tự, an toàn giao thông năm 2023. Technical report, National Traffic Safety Committee of Vietnam, 2023. Báo cáo thường niên về an toàn giao thông Việt Nam.